



Вычислительная геометрия и алгоритмы компьютерной графики

Лекция №2: Размещение объектов на сцене и проекция на экран

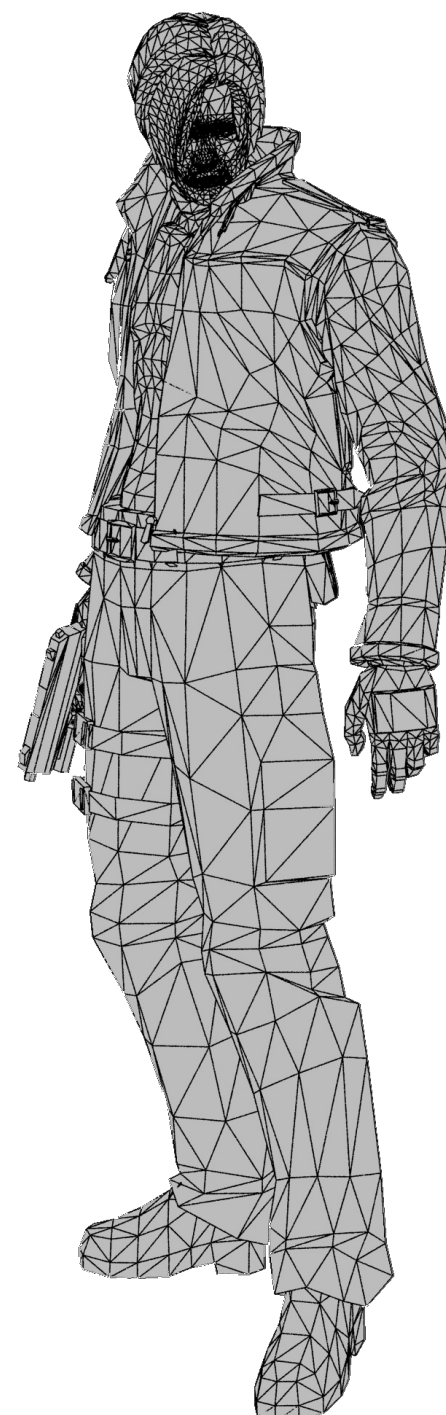
к.ф.-м.н., доцент

Рябинин Константин Валентинович,
kostya.ryabinin@gmail.com

ст. преподаватель

Лабутин Иван Александрович,
linuxrf@gmail.com

Объекты в полигональной 3D-графике представляют собой поверхности, аппроксимированные множеством многоугольников

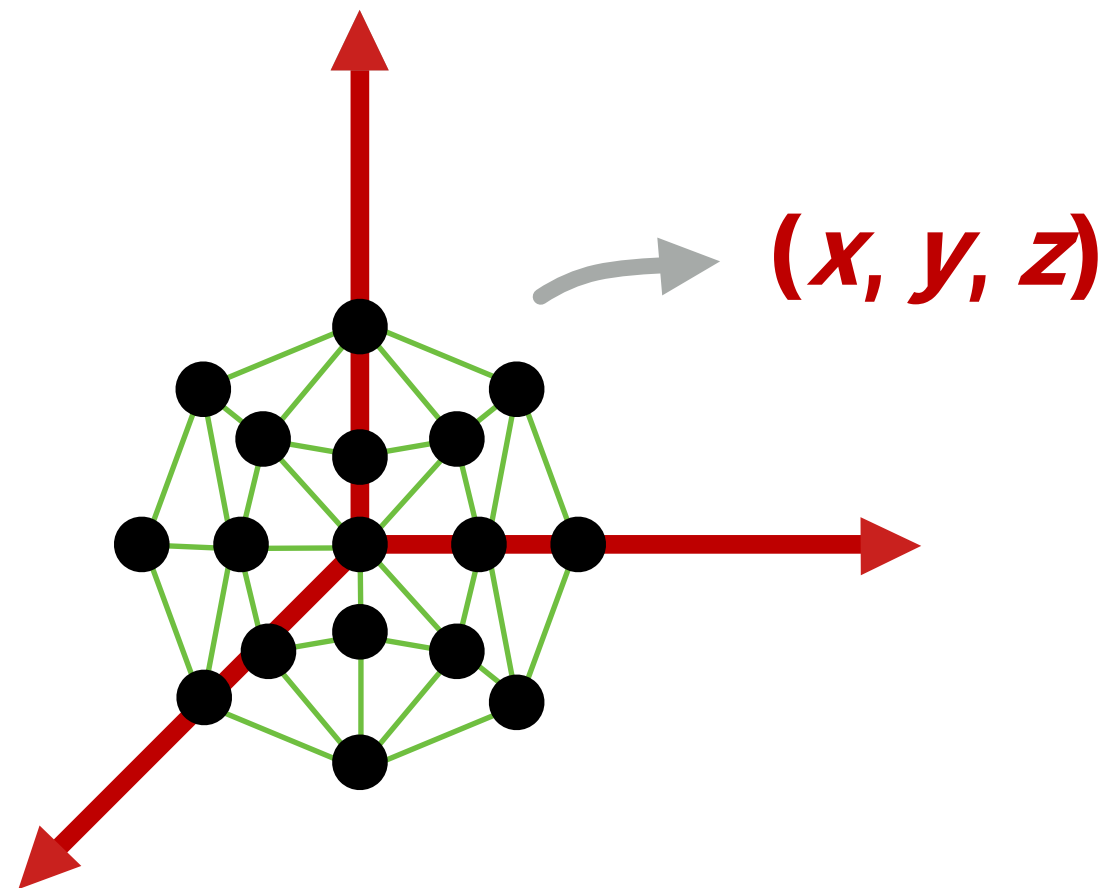


- Атомарная управляемая единица геометрии – **вершина**
- Вершина имеет набор атрибутов (типа float), интерпретация которых лежит на программисте:
 - Координаты в пространстве
 - Координаты нормали
 - Координаты текстуры
 - Цвет
 - ...
- Вершины объединяются в примитивы, почти всегда – **треугольники**

- Сцена состоит из **объектов**
- Объекты состоят из **примитивов**
- Примитивы состоят из **вершин**
- Вершины имеют **3D-координаты**

1. Разместить объекты в пространстве
2. Спроецировать их примитивы на плоскость экрана
3. Подобрать для проекций пиксели (**растеризация**)
4. Раскрасить пиксели

Каждый объект имеет **локальную** систему координат



Все типовые задачи «размещения» объектов решаются при помощи аффинных преобразований вершин этих объектов:

- 1. Параллельного переноса**
- 2. Масштабирования**
- 3. Поворота**
- 4. Наклона**

Аффинные преобразования – отображение пространства в себя, при котором параллельные прямые переходят в параллельные прямые:

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^n$$
$$f(x) = M \cdot x + v$$

где M – матрица аффинного преобразования размерности n , $v \in \mathbb{R}^n$

Иначе говоря, преобразование называется аффинным, если его можно получить следующим образом:

- **Выбрать «новый» базис пространства с «новым» началом координат v**
- **Каждой точке x пространства поставить в соответствие точку $f(x)$, имеющую те же координаты относительно «новой» системы координат, что и x в «старой».**

- В трёхмерной компьютерной графике преобразования производятся в трёхмерном пространстве (ваш К.О.)
- Помимо аффинных преобразований, используются ещё преобразования проекции
- С целью упрощения формулы и унификации аффинных и проективных преобразований, переходят к **однородным** координатам векторов и матрицам размерности 4x4:

$$f(x) = M \cdot x$$

При использовании матриц все преобразования сводятся к умножению вектора однородных координат вершины на матрицу преобразования, в результате чего получается вектор «новых» координат данной вершины:

$$\begin{pmatrix} m_0 & m_4 & m_8 & m_{12} \\ m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$

ось X новой системы координат
ось Y новой системы координат
ось Z новой системы координат
начало новой системы координат

Матрица переноса

$$\begin{pmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Матрица масштаба

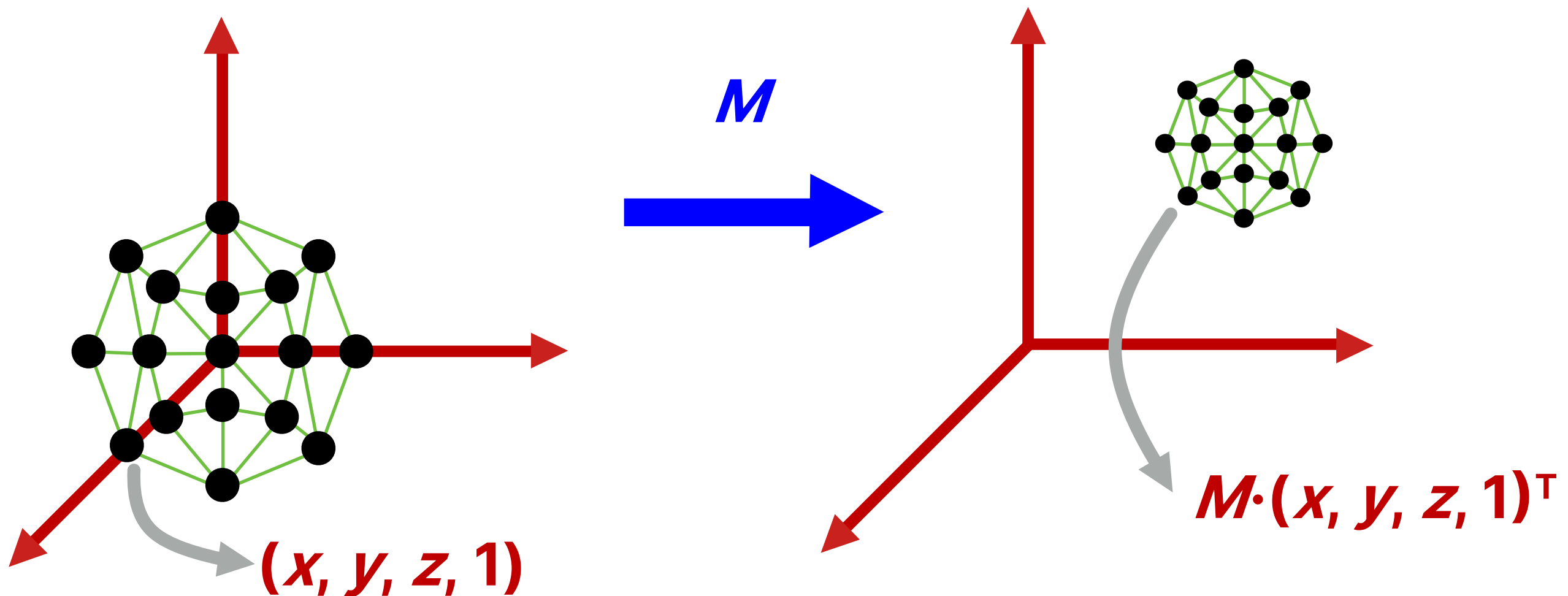
$$\begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Матрица поворота

$$\begin{pmatrix} x^2(1-c)+c & xy(1-c)-zs & xz(1-c)+ys & 0 \\ yx(1-c)+zs & y^2(1-c)+c & yz(1-c)-xs & 0 \\ xz(1-c)-ys & yz(1-c)+xs & z^2(1-c)+c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$c = \cos \theta \quad s = \sin \theta \quad |(x, y, z)| = 1$$

Аффинные преобразования позволяют перейти от локальной системы координат к **глобальной**



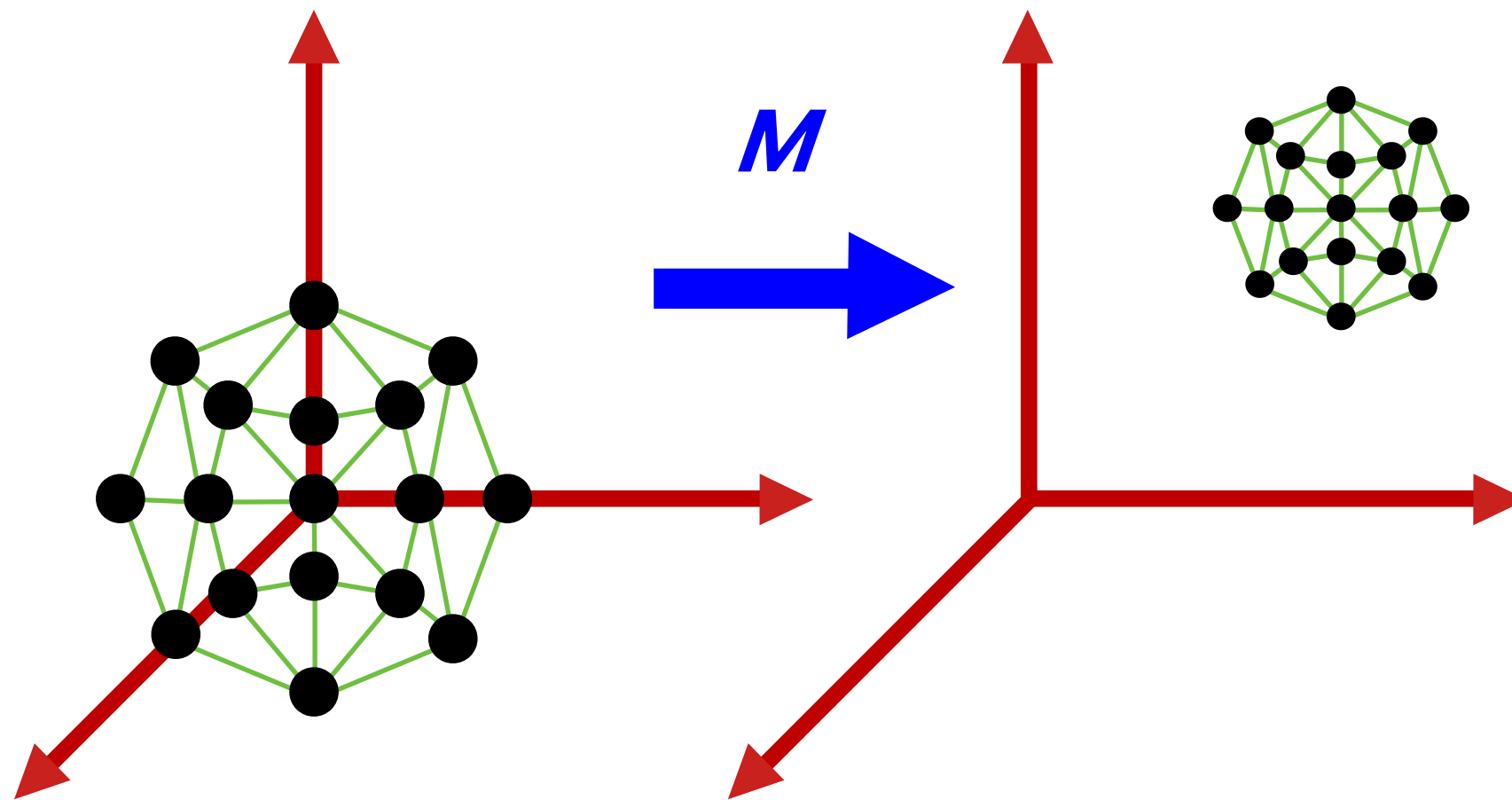
- Суперпозиция аффинных преобразований осуществляется через **перемножение** соответствующих матриц
- Суперпозиция некоммутативна, но ассоциативна:

$$M_1 \cdot M_2 \neq M_2 \cdot M_1$$

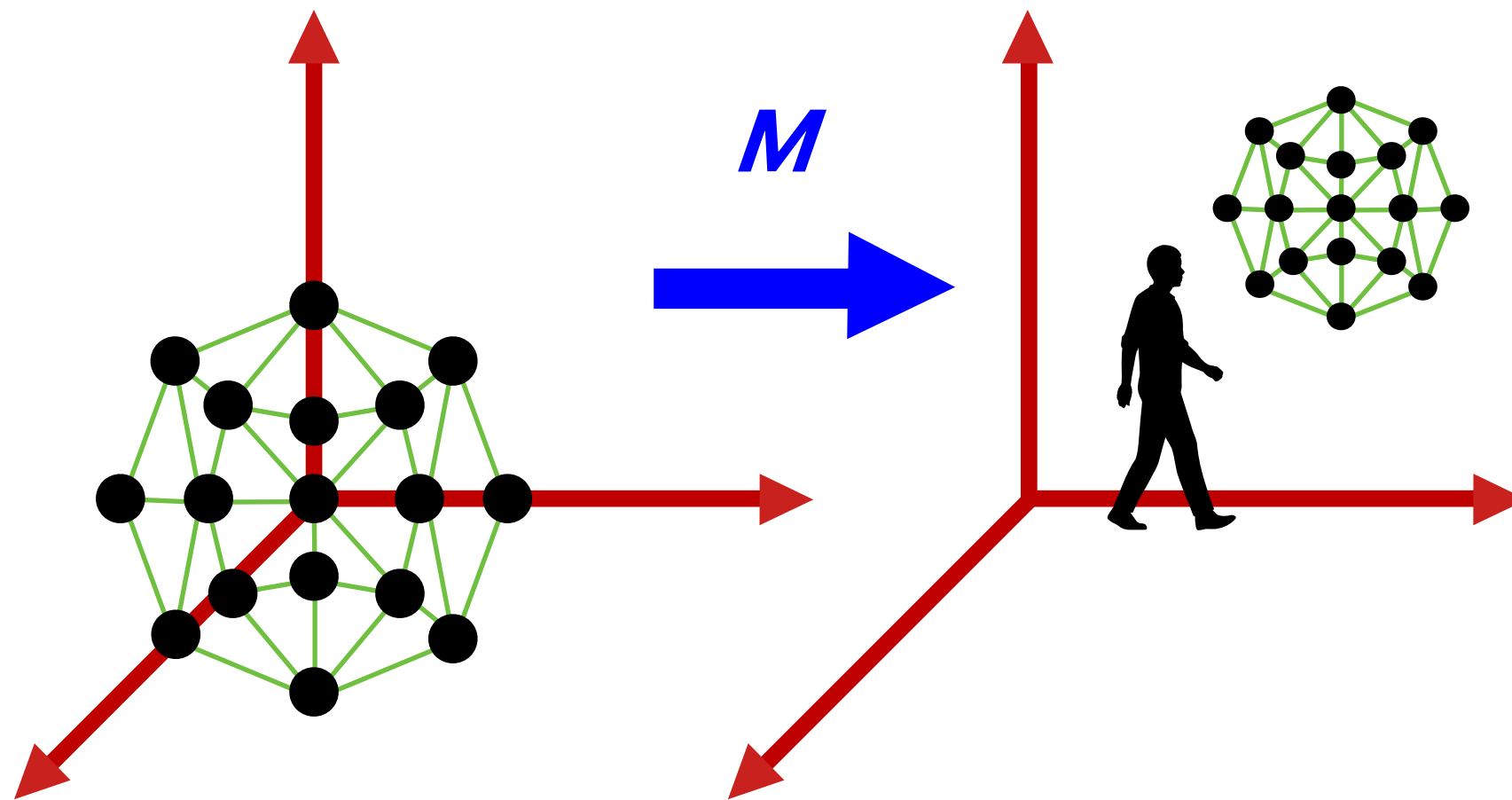
$$(M_1 \cdot M_2) \cdot M_3 = M_1 \cdot (M_2 \cdot M_3)$$

- Обратная матрица соответствует обратному аффинному преобразованию
- Интерпретация цепочки преобразований:
 - Читаем цепочку справа налево
 - Каждая следующая матрица в цепочке – «команда» объекту совершить то или иное действие относительно его текущего положения

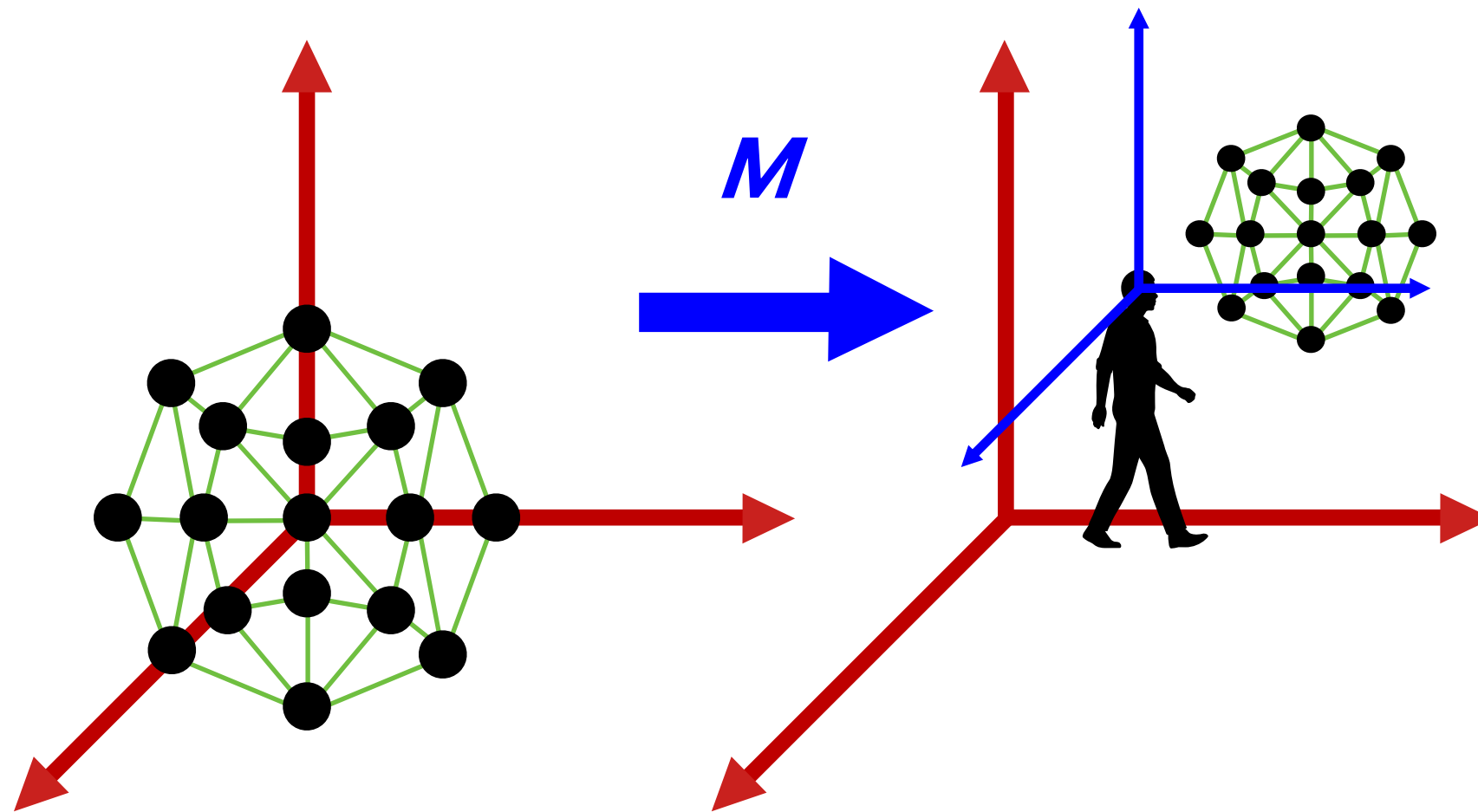
От глобальной системы можно опционально перейти к системе **наблюдателя**



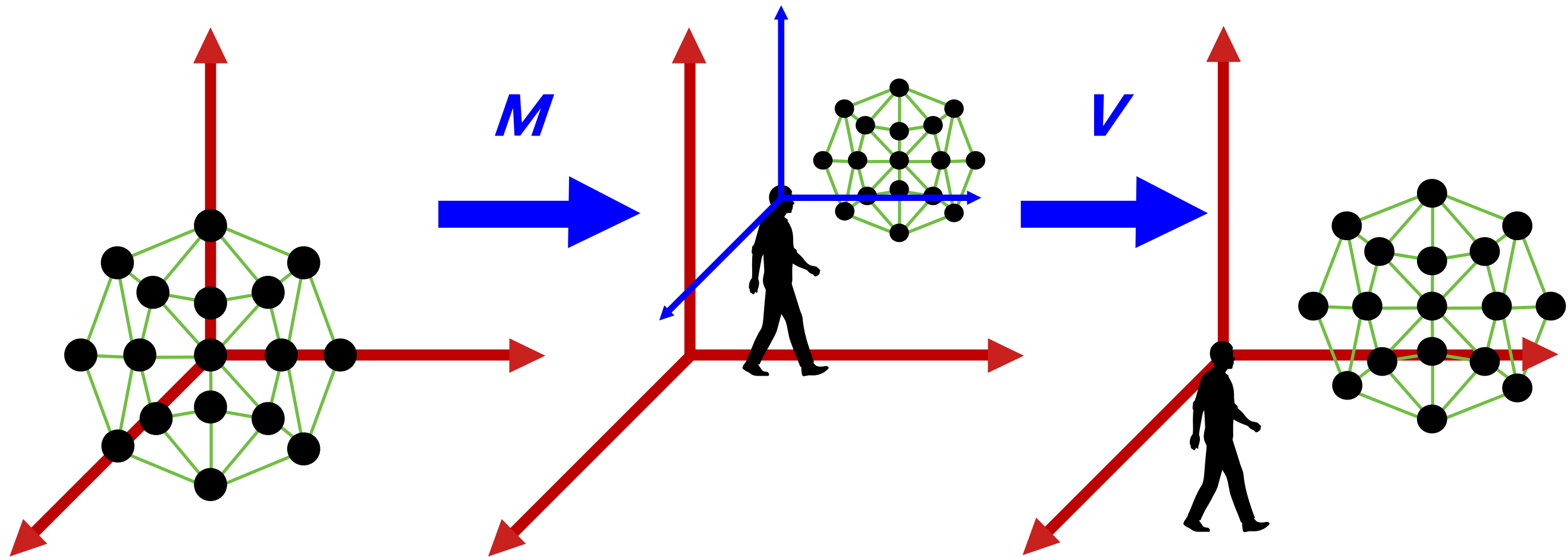
От глобальной системы можно опционально перейти к системе **наблюдателя**



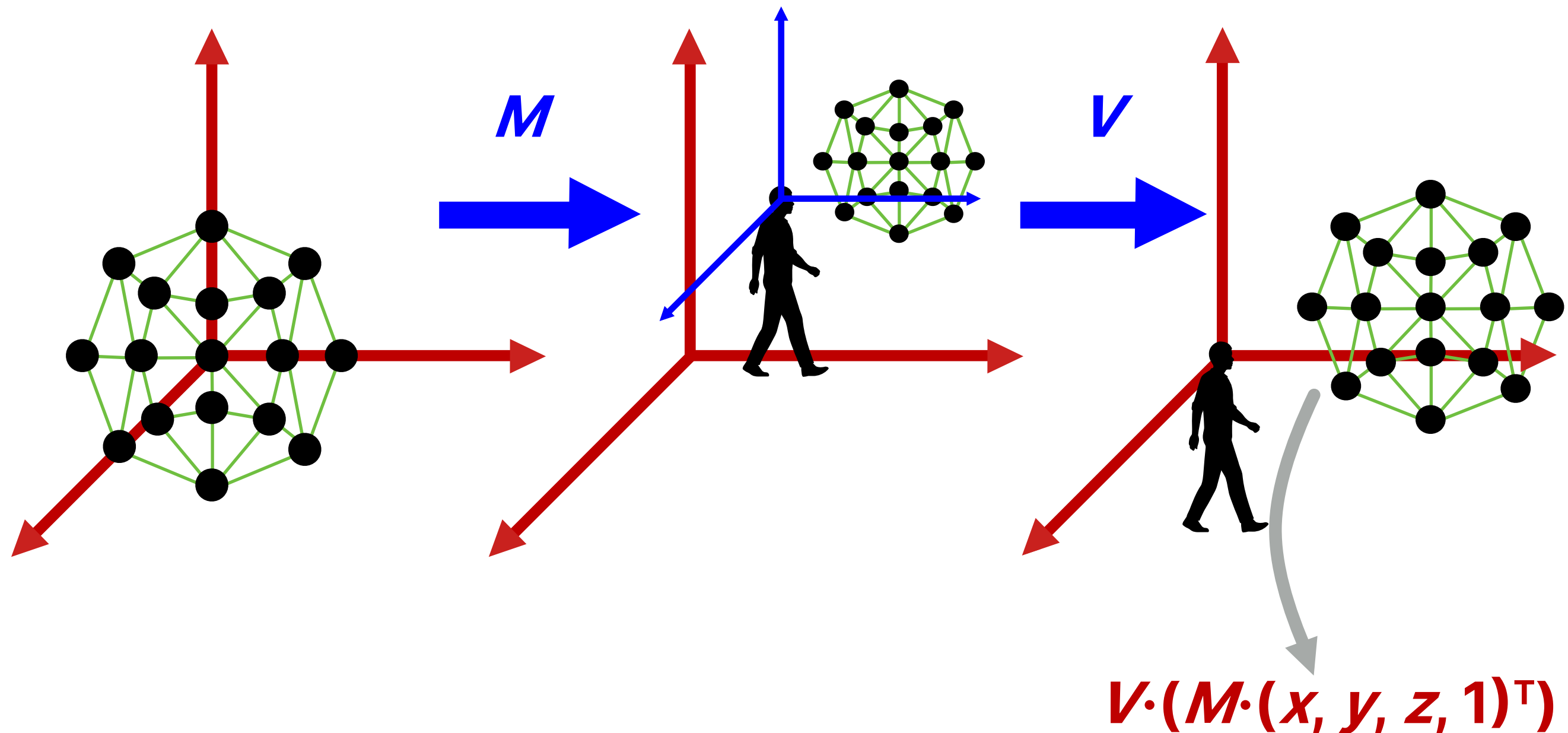
От глобальной системы можно опционально перейти к системе **наблюдателя**



От глобальной системы можно опционально перейти к системе **наблюдателя**



От глобальной системы можно опционально перейти к системе **наблюдателя**



Экран – это где?!

Экран – это где?!

Экран там, где NDC

Экран – это где?!

Экран там, где NDC

NDC – нормированные координаты устройства

Экран – это где?!

Экран там, где NDC

NDC – нормированные координаты устройства

Экран – это где?!

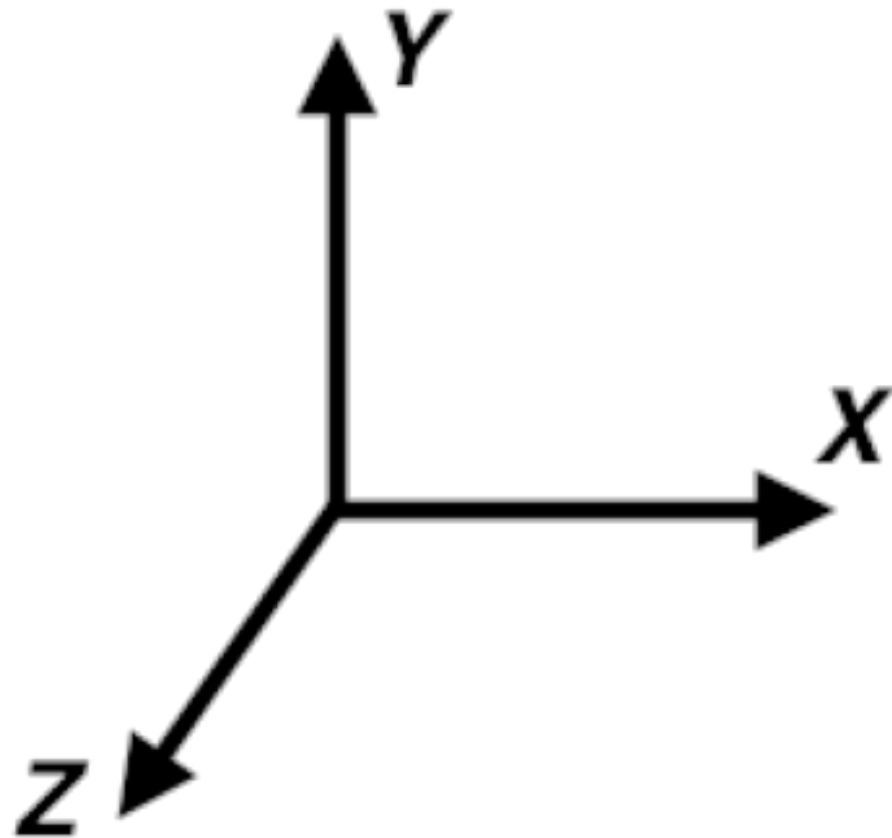
Экран там, где NDC

NDC – нормированные координаты устройства

Главное правило:

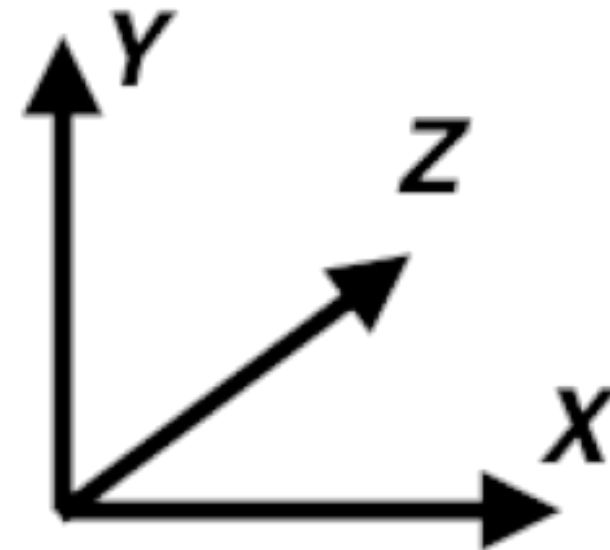
ВИДИМО ТО, И ТОЛЬКО ТО, ЧТО ПОПАЛО В NDC

Конкретный вид NDC – конвенционализм



OpenGL

$[-1; 1] \times [-1; 1] \times [-1; 1]$

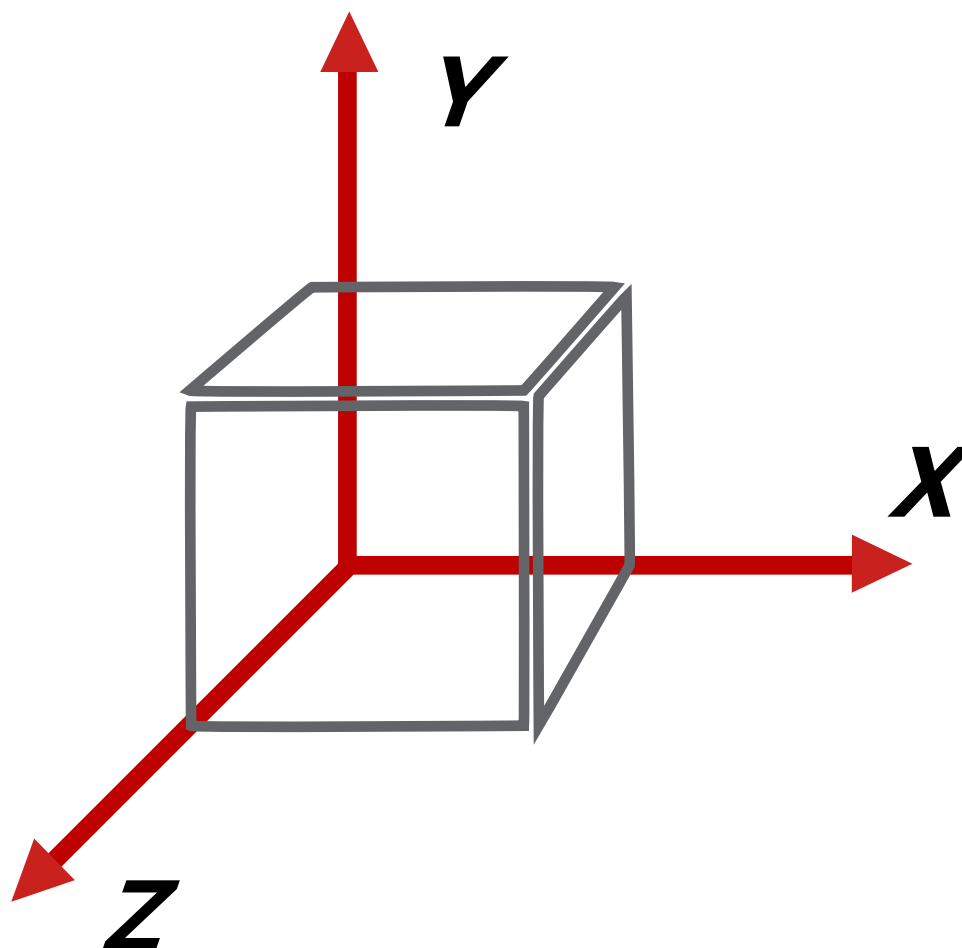


DirectX

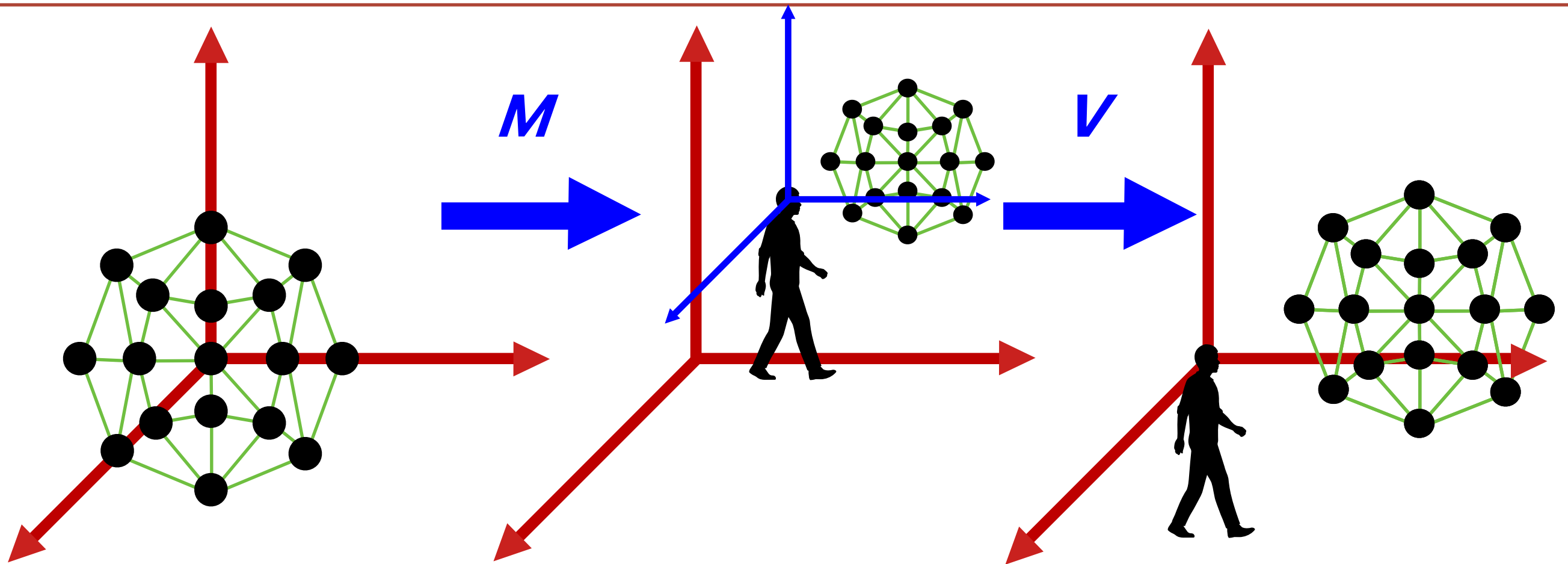
$[-1; 1] \times [-1; 1] \times [0; 1]$

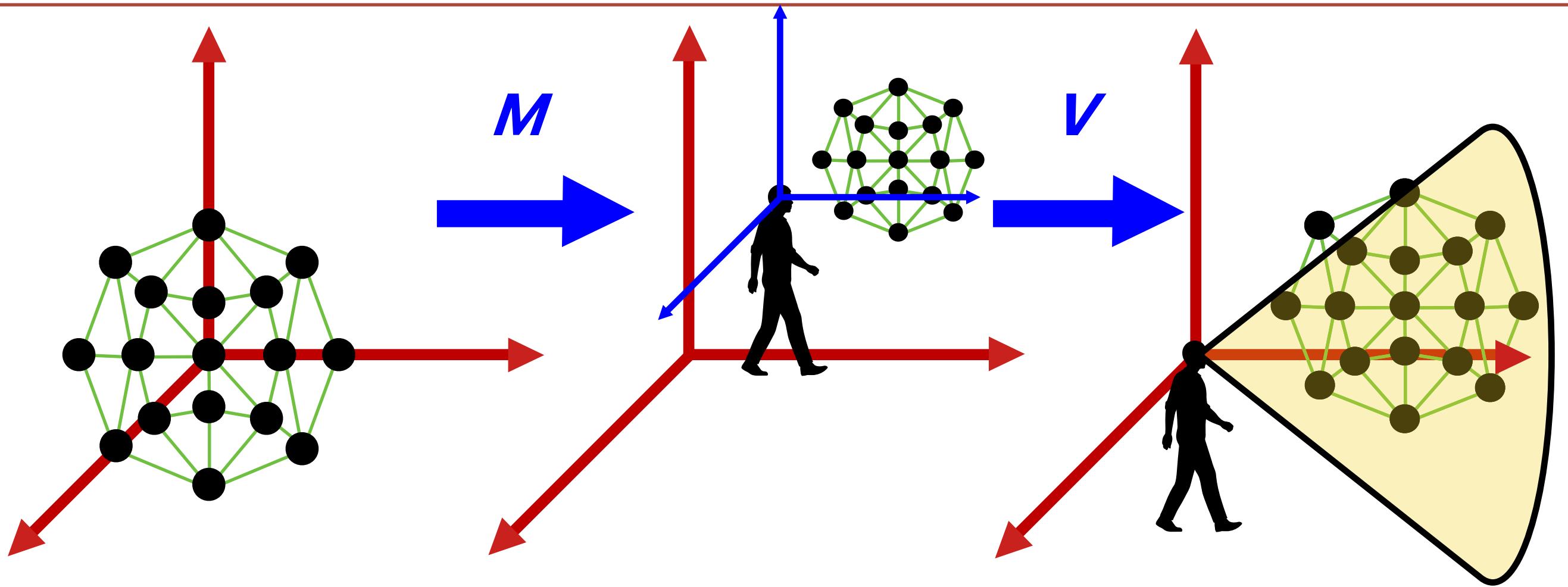
Иногда программисту дается возможность определения параметров NDC

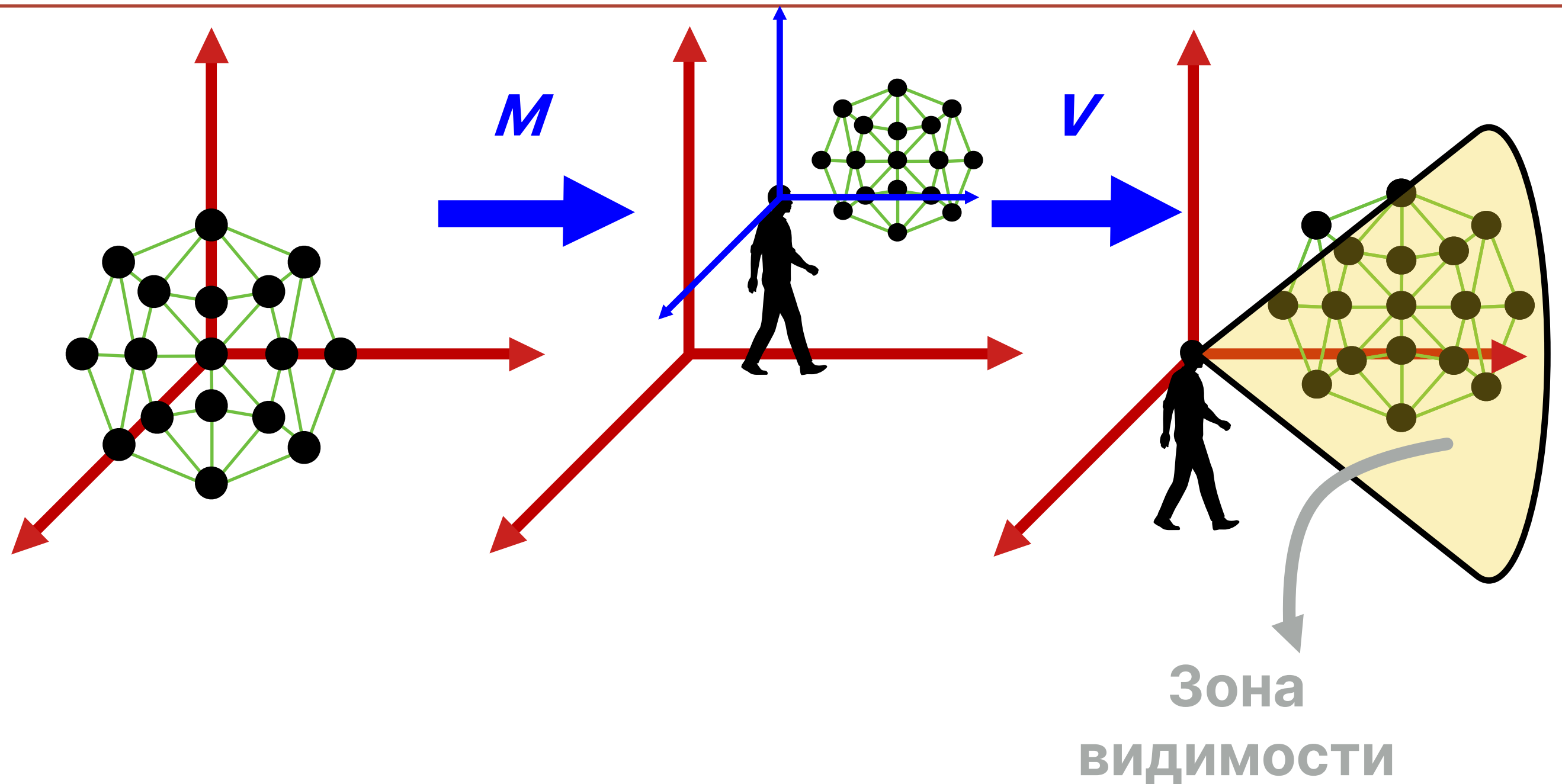
NDC в OpenGL

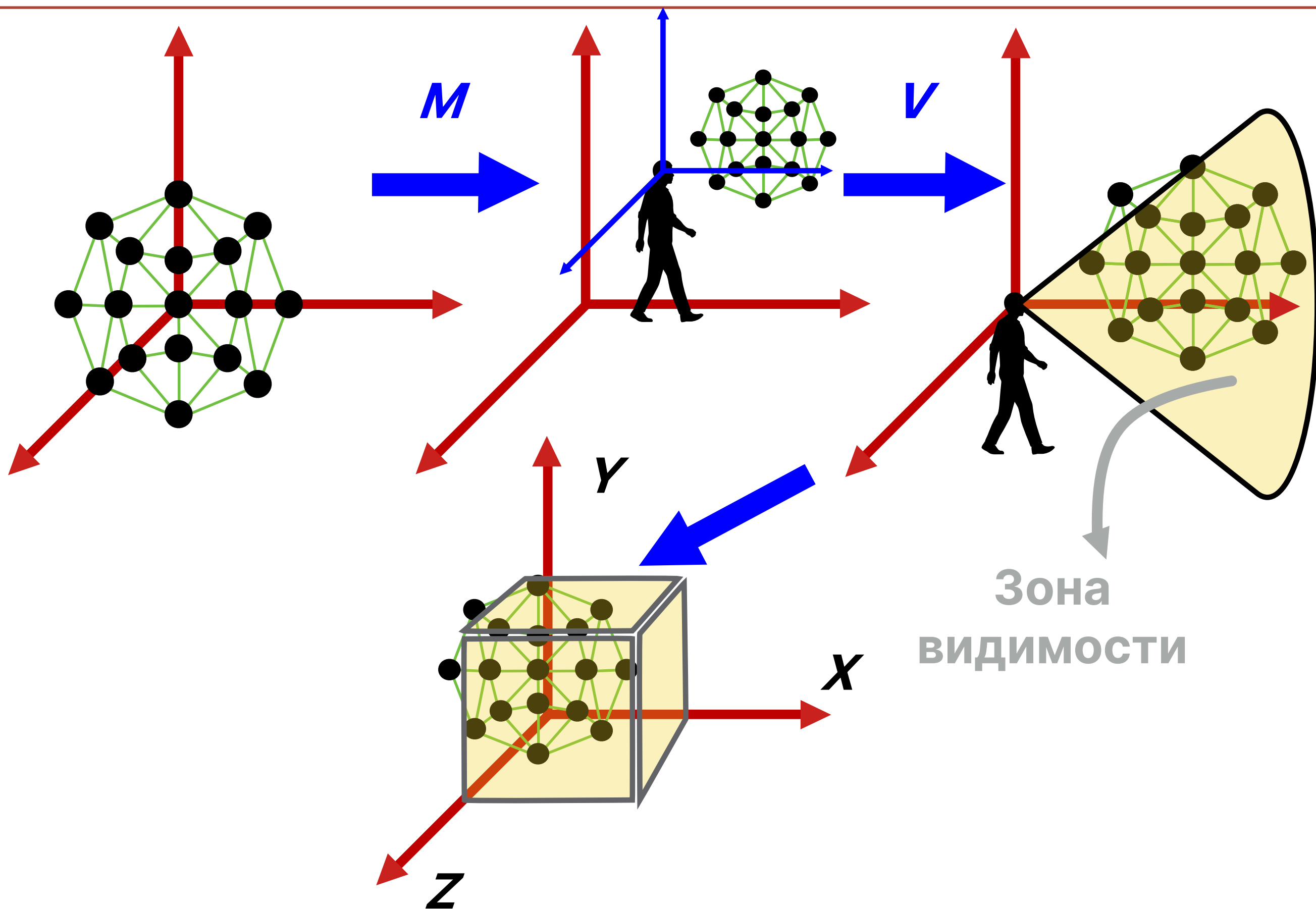


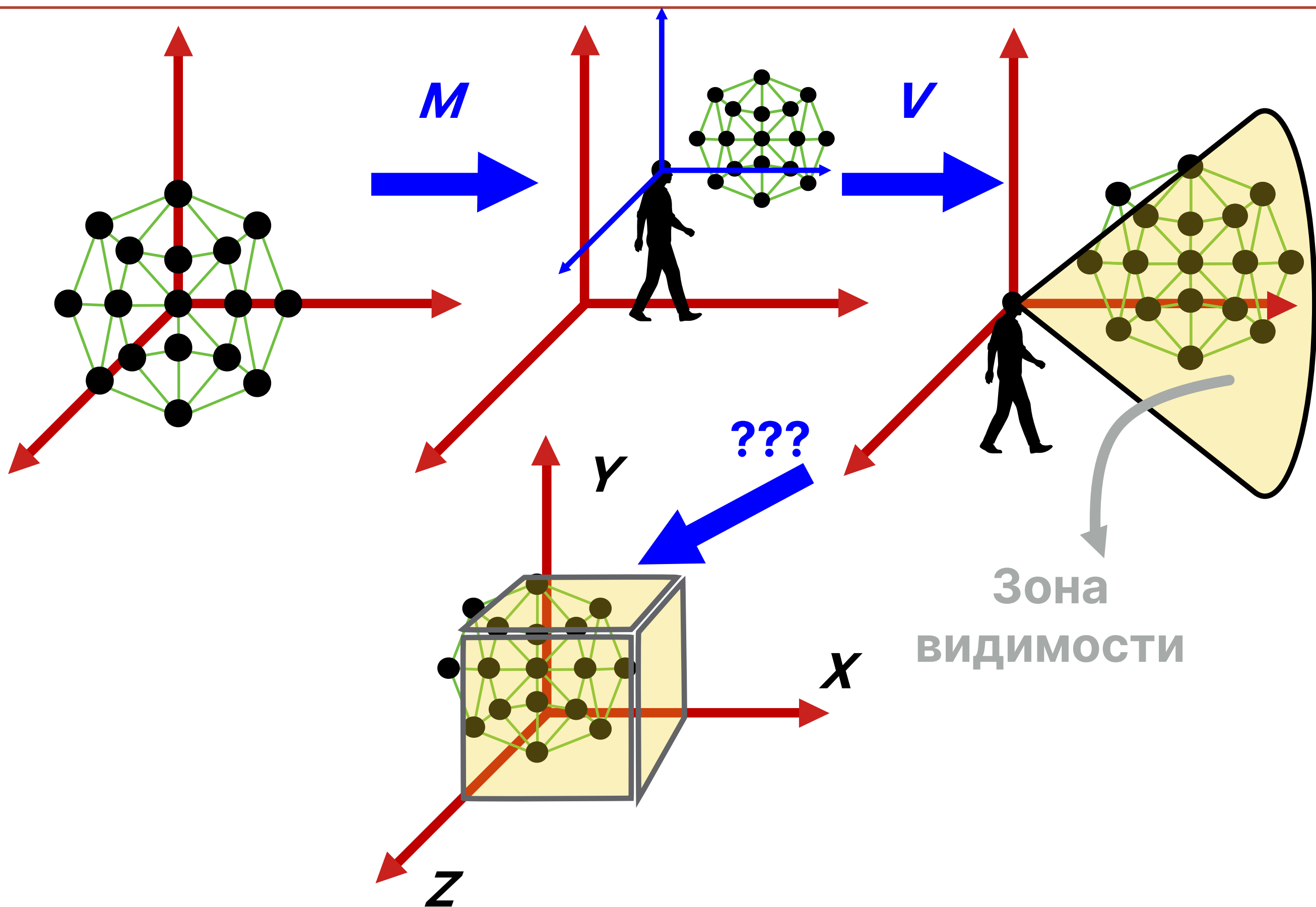
Но работать напрямую в этой системе координат – почти всегда неудобно



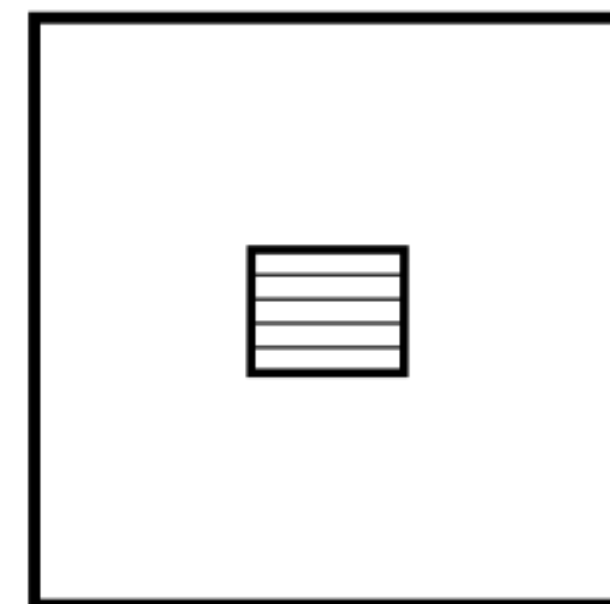
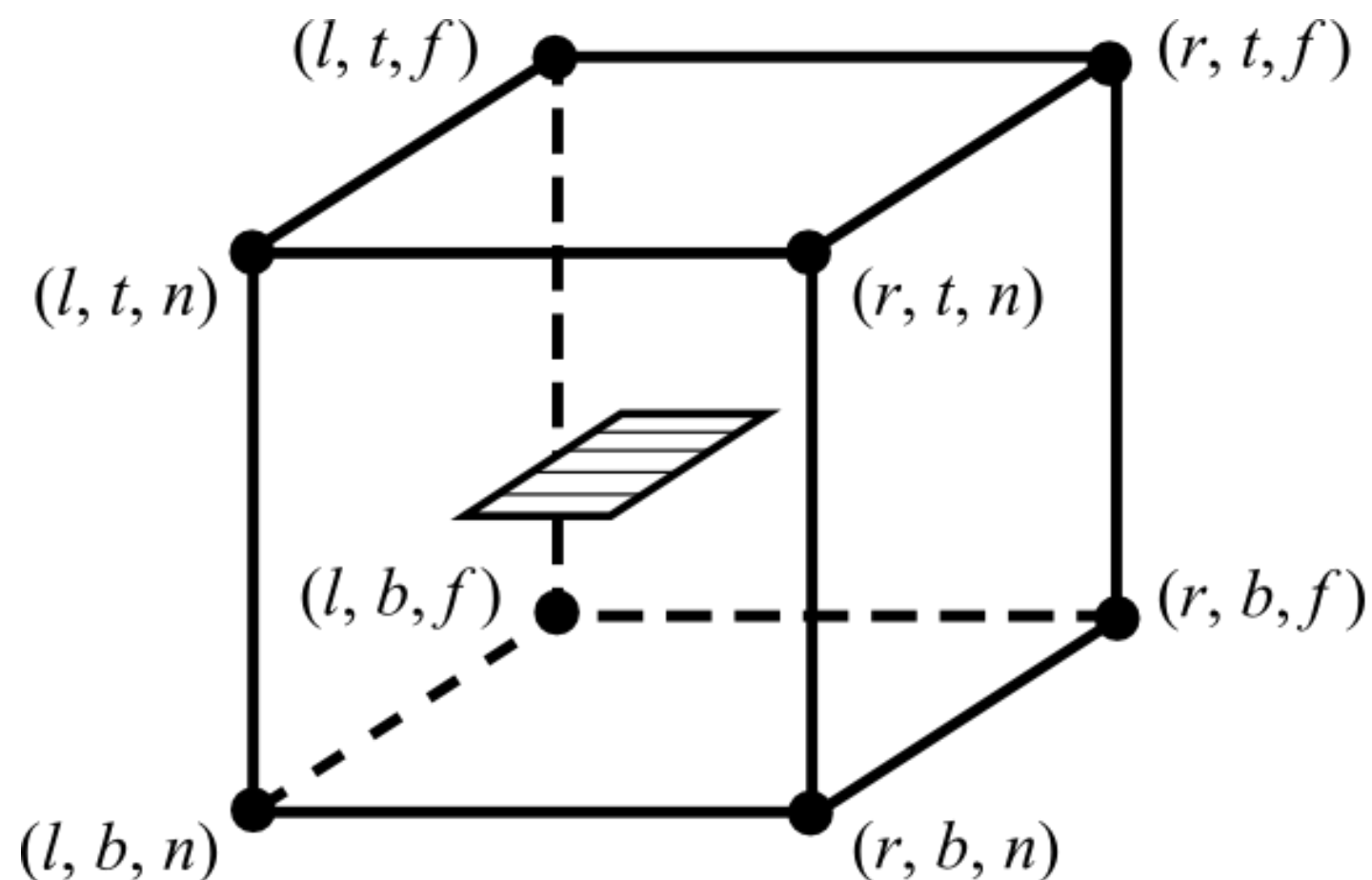




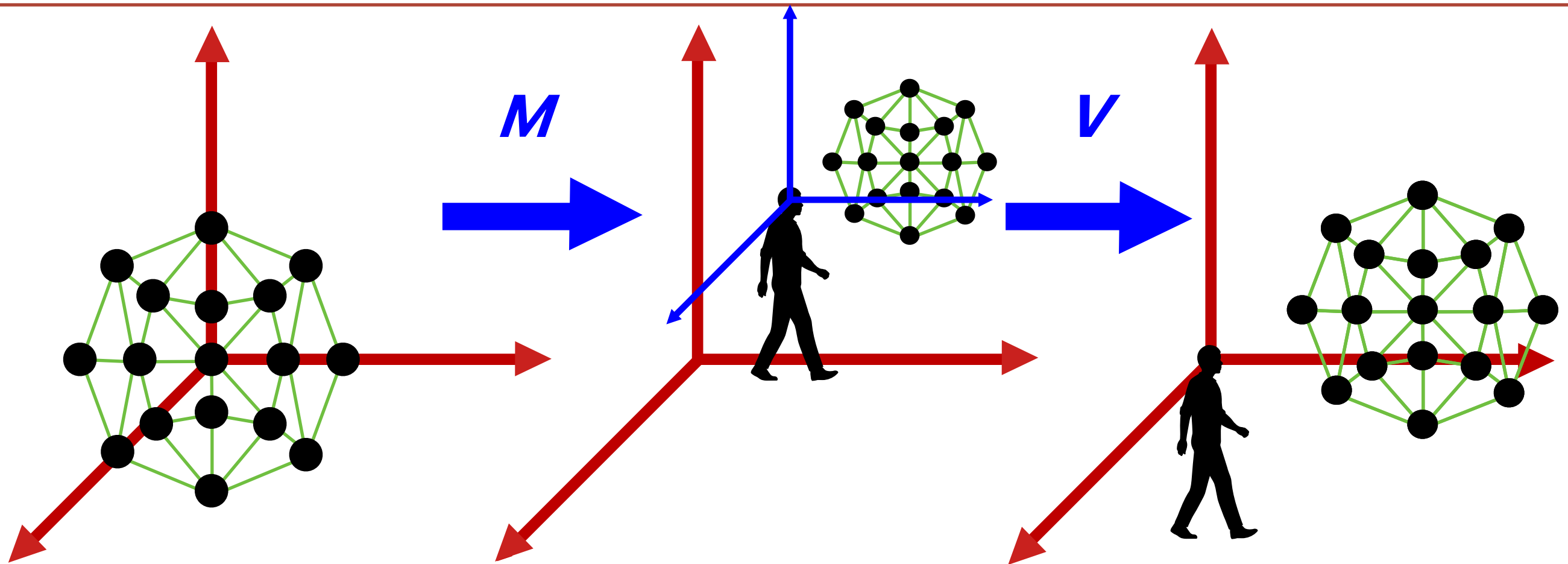


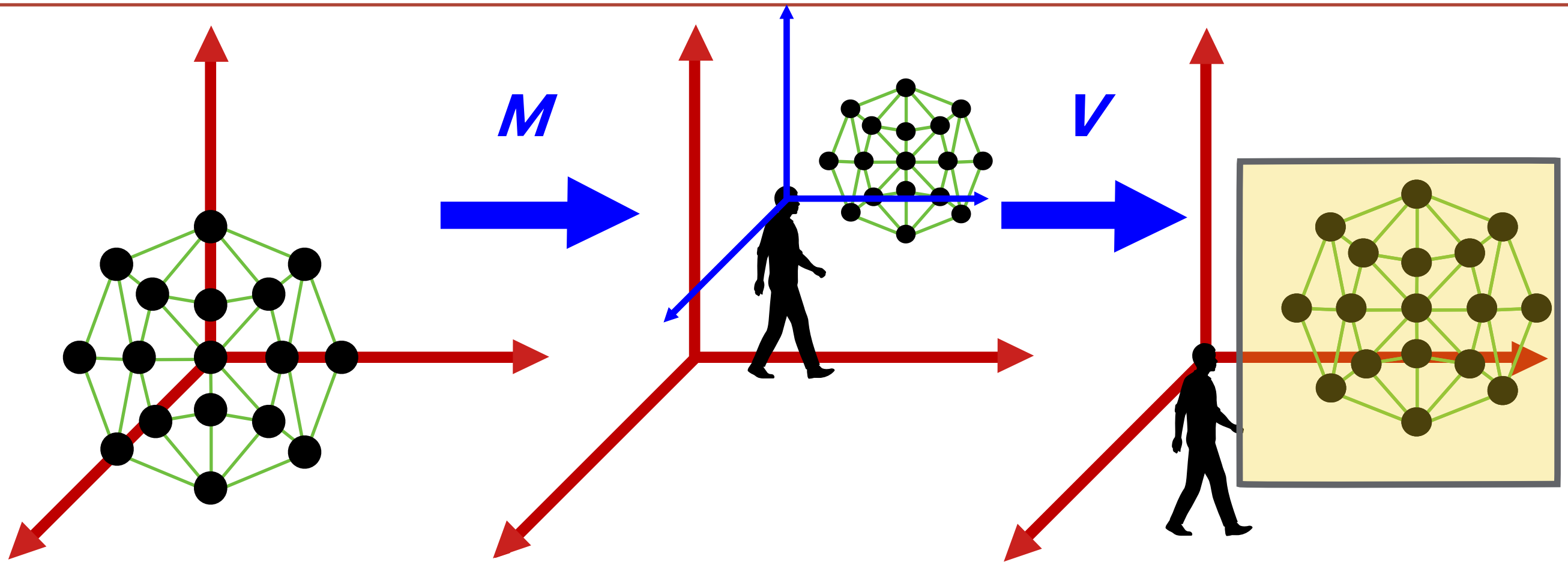


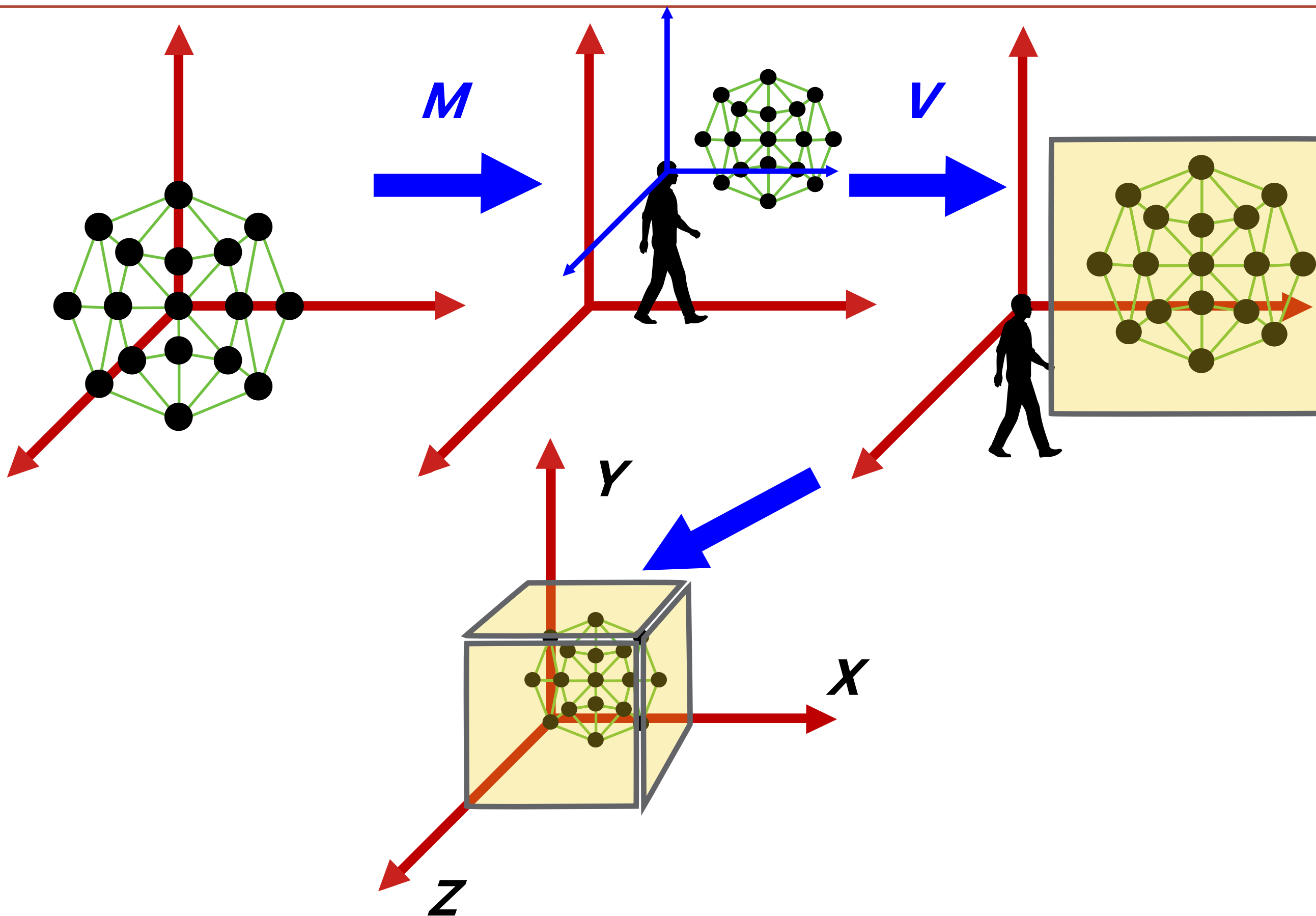
- Преобразование области видимости в NDC для единообразия также осуществляется при помощи матриц
- Матрица преобразования в данном случае носит название **матрицы проекции**, хотя само умножение и не изменяет размерность вектора
- Используются **параллельные** и **перспективные** проекции

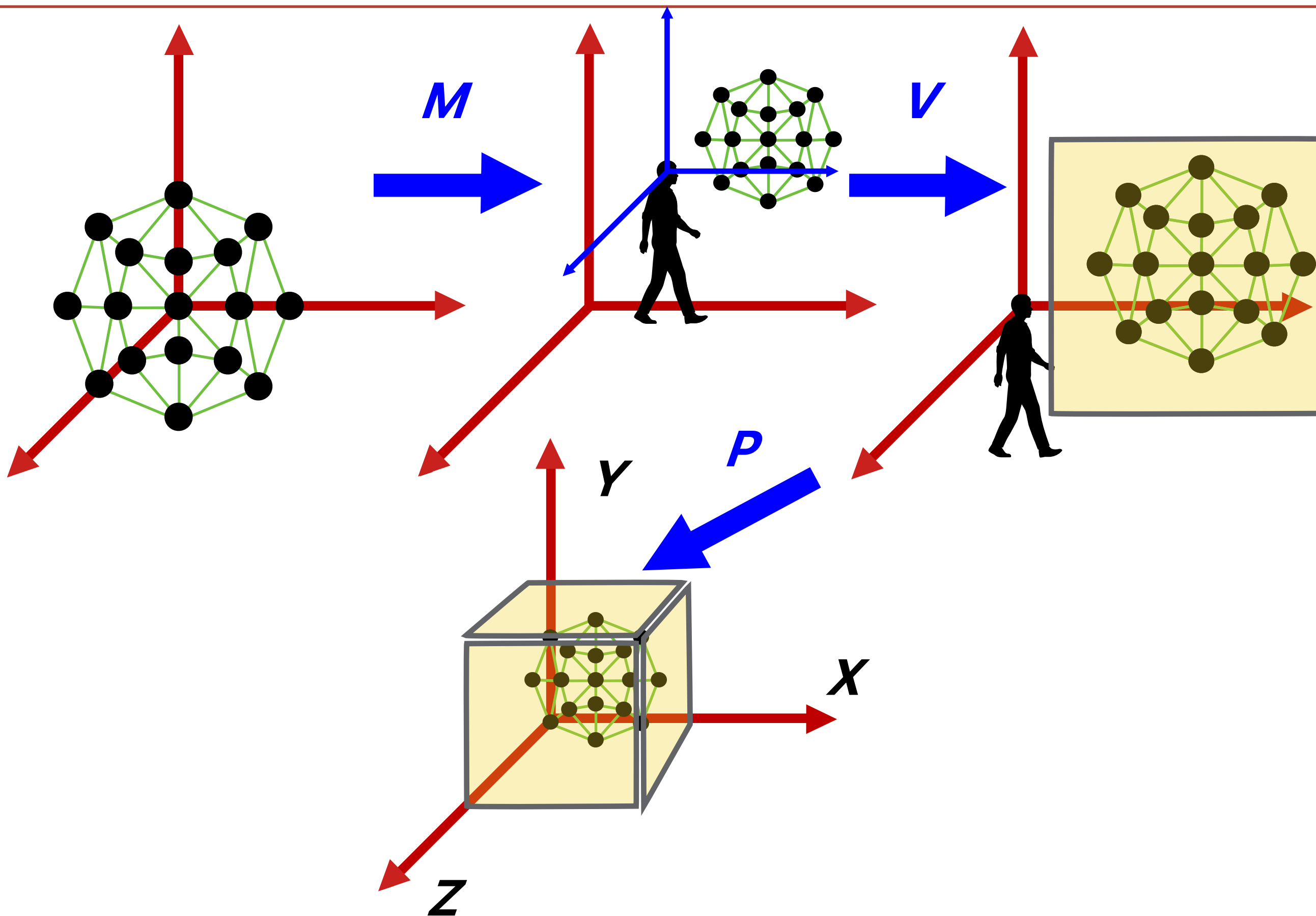


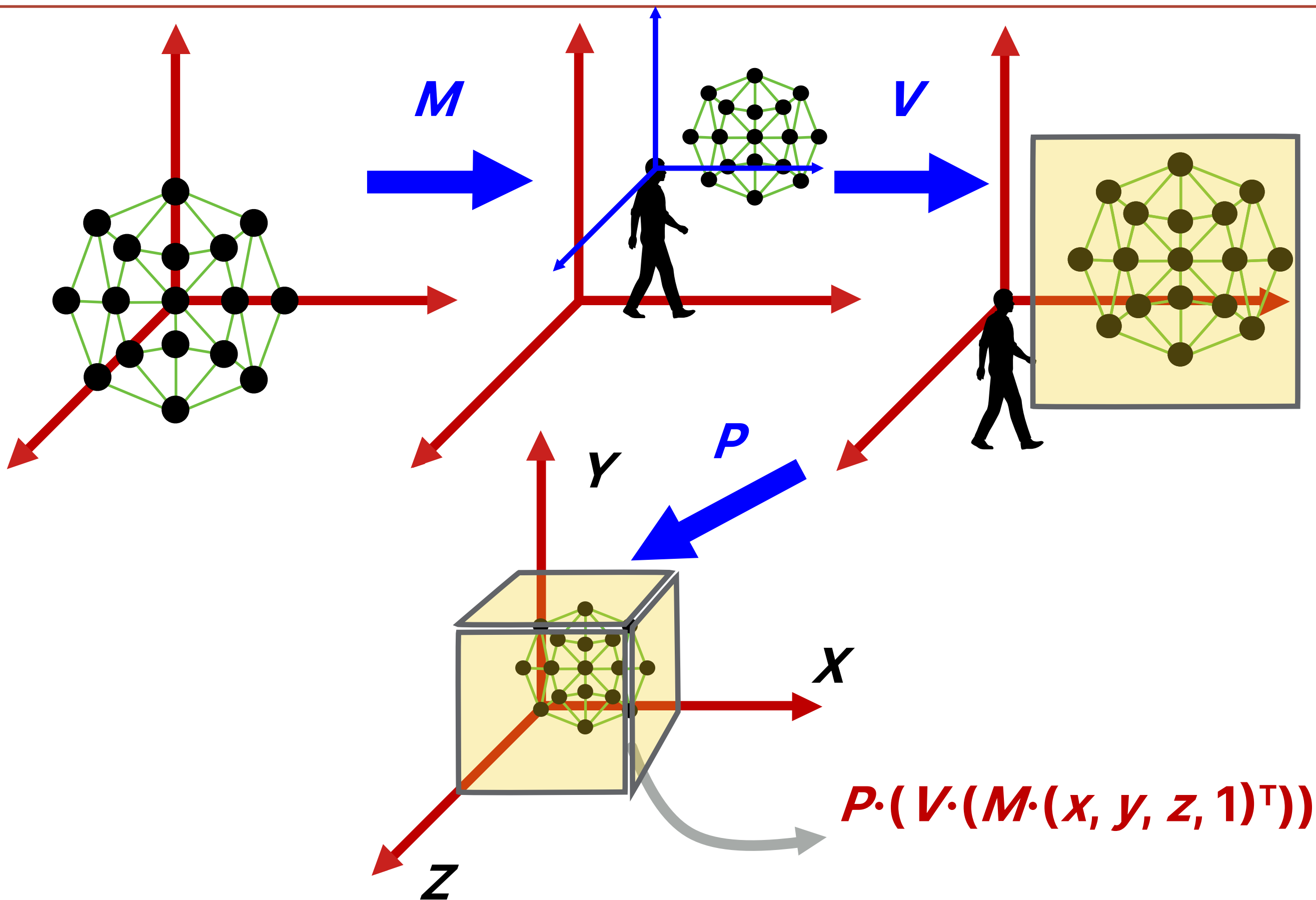
$$\begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

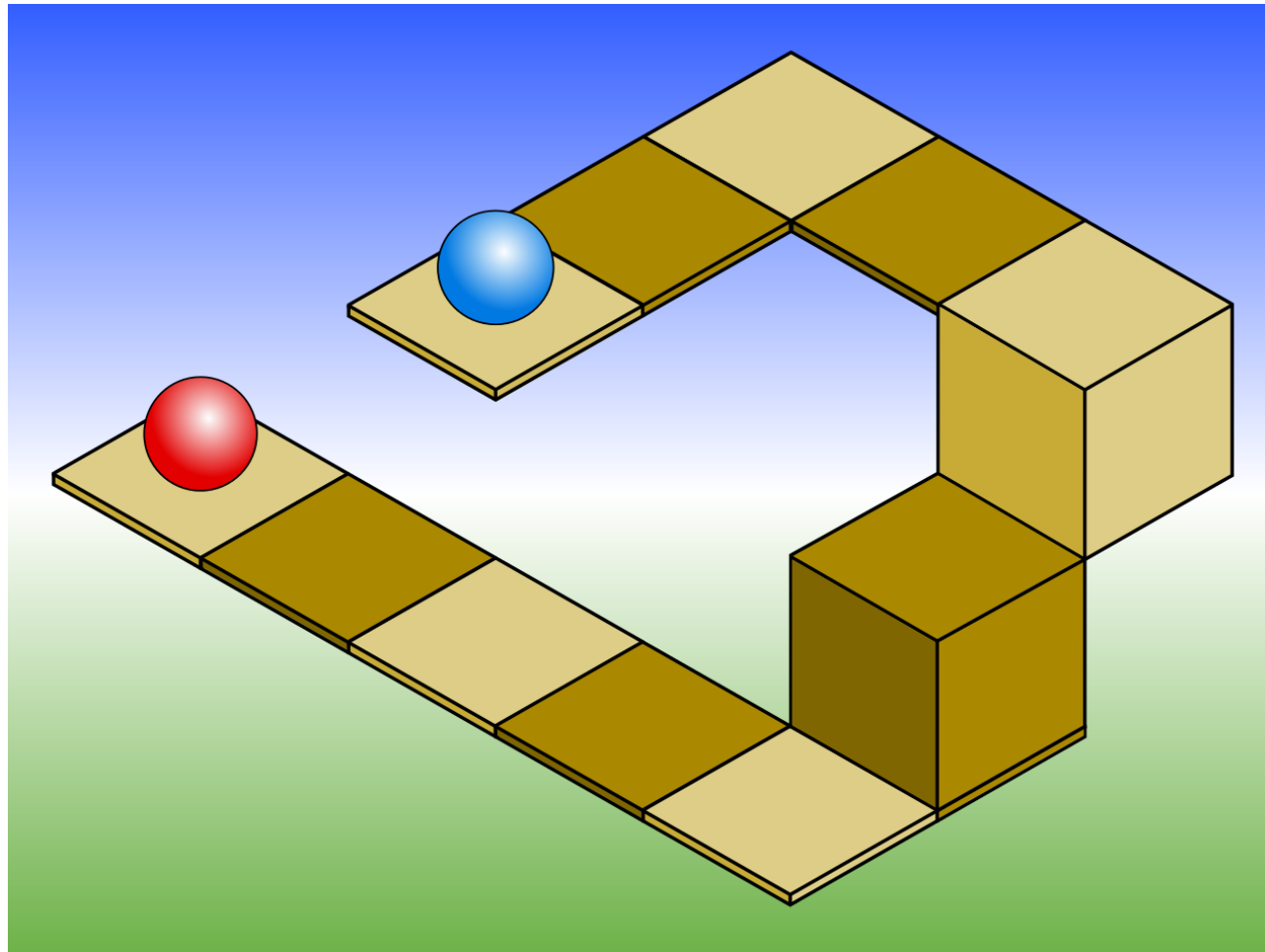


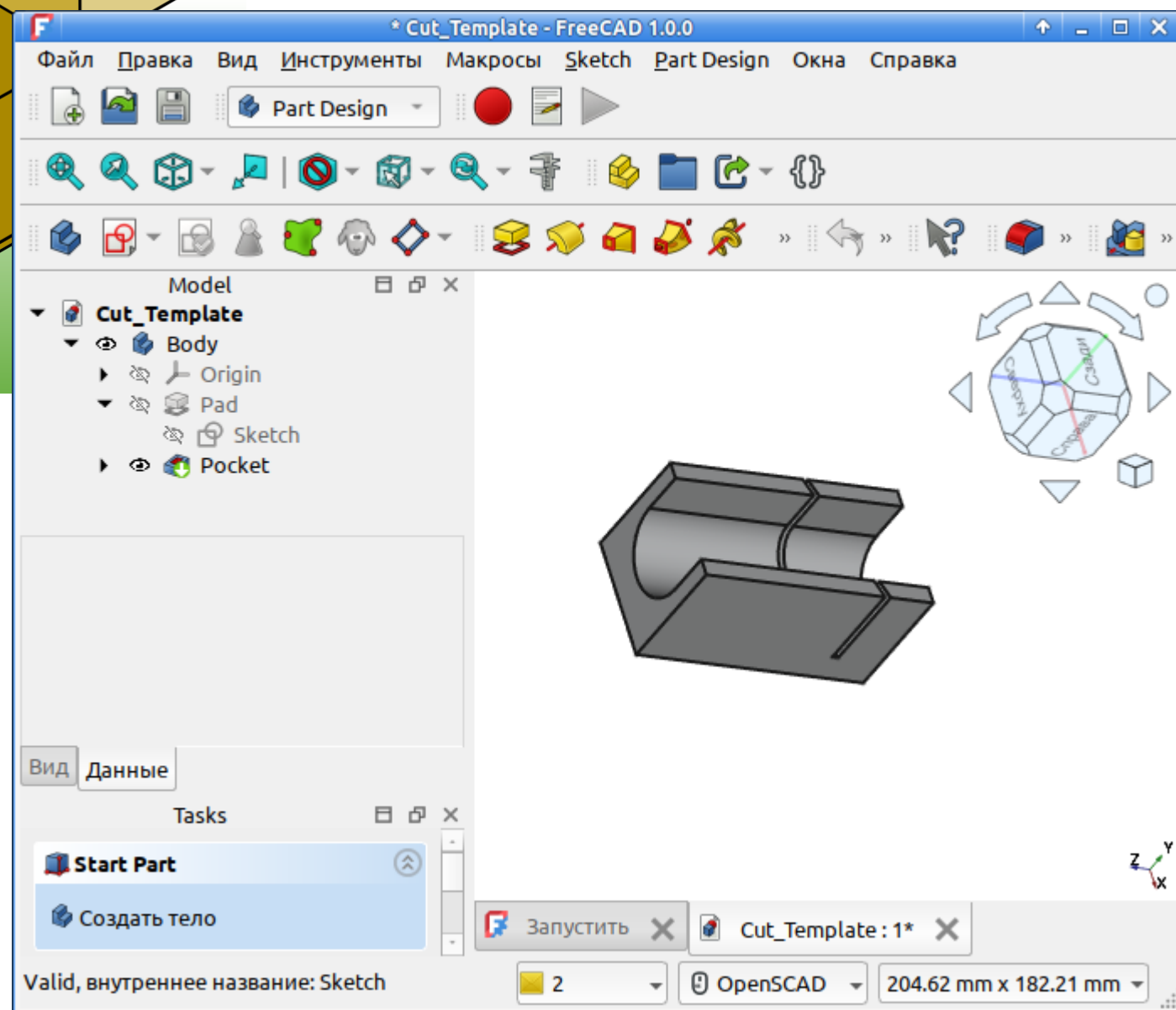
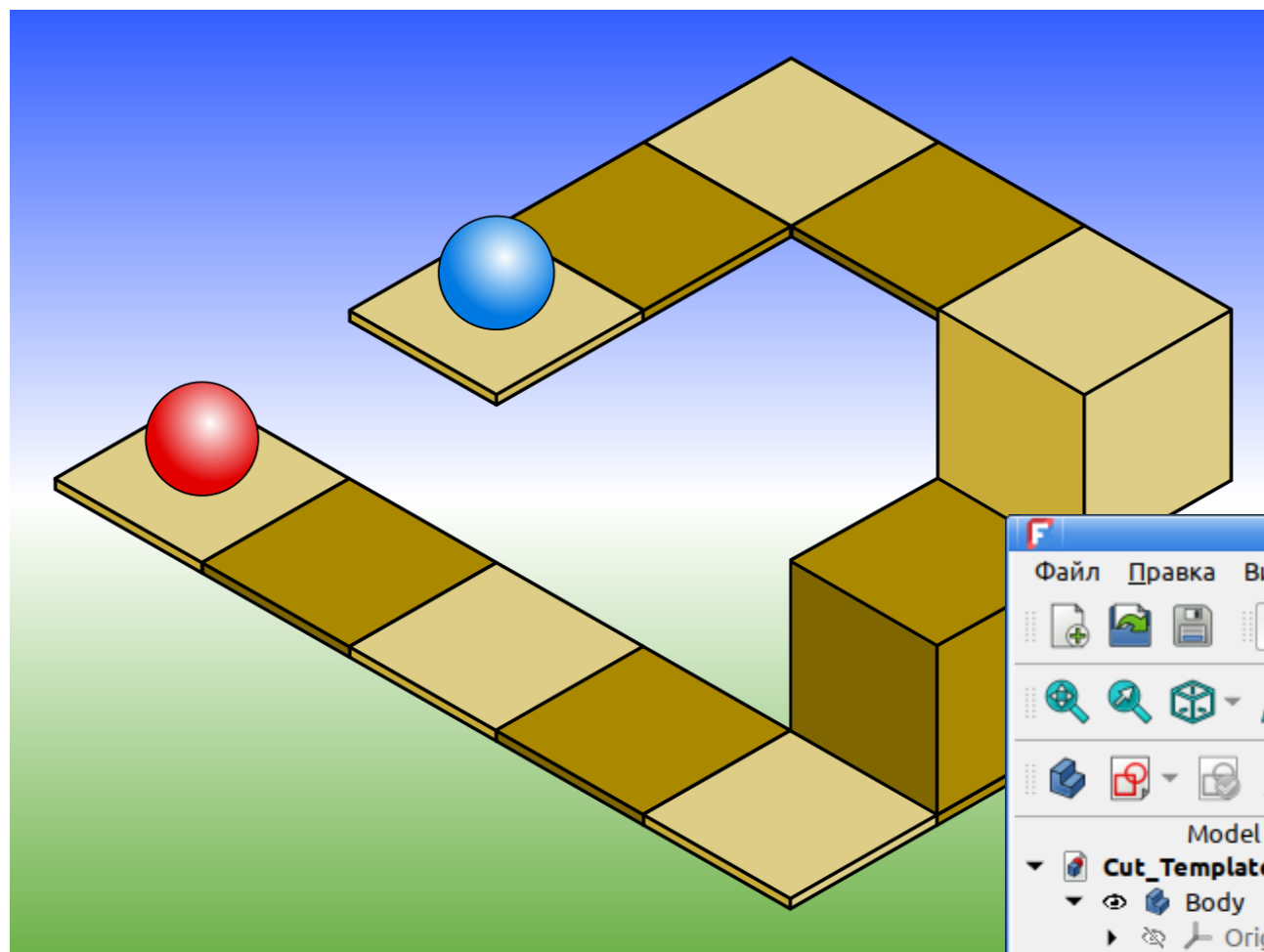




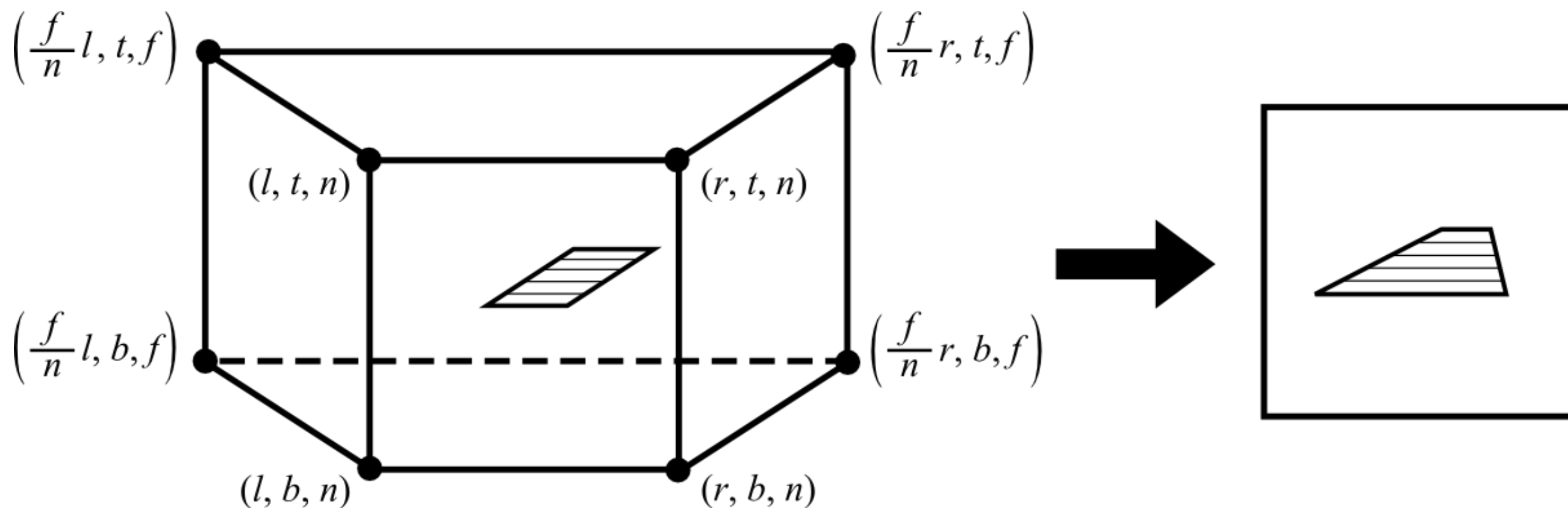




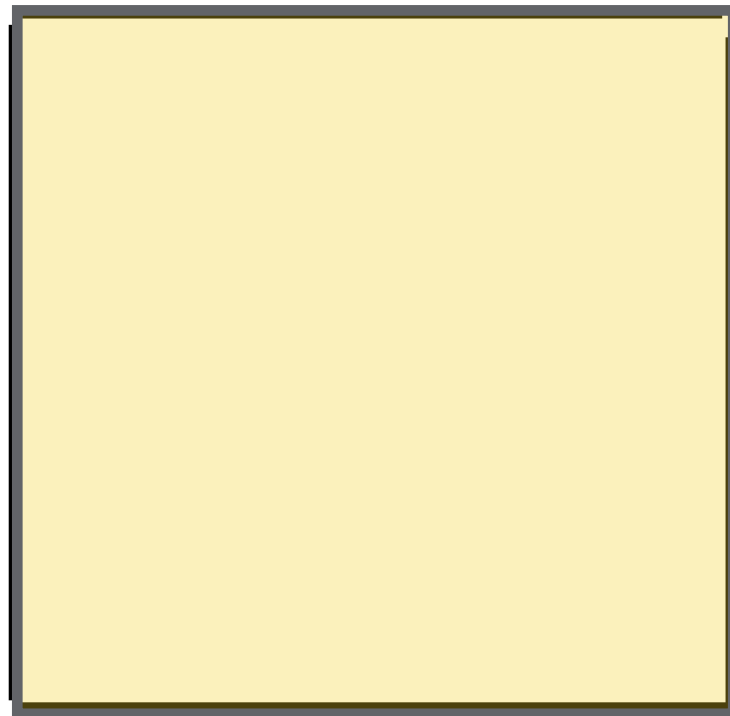




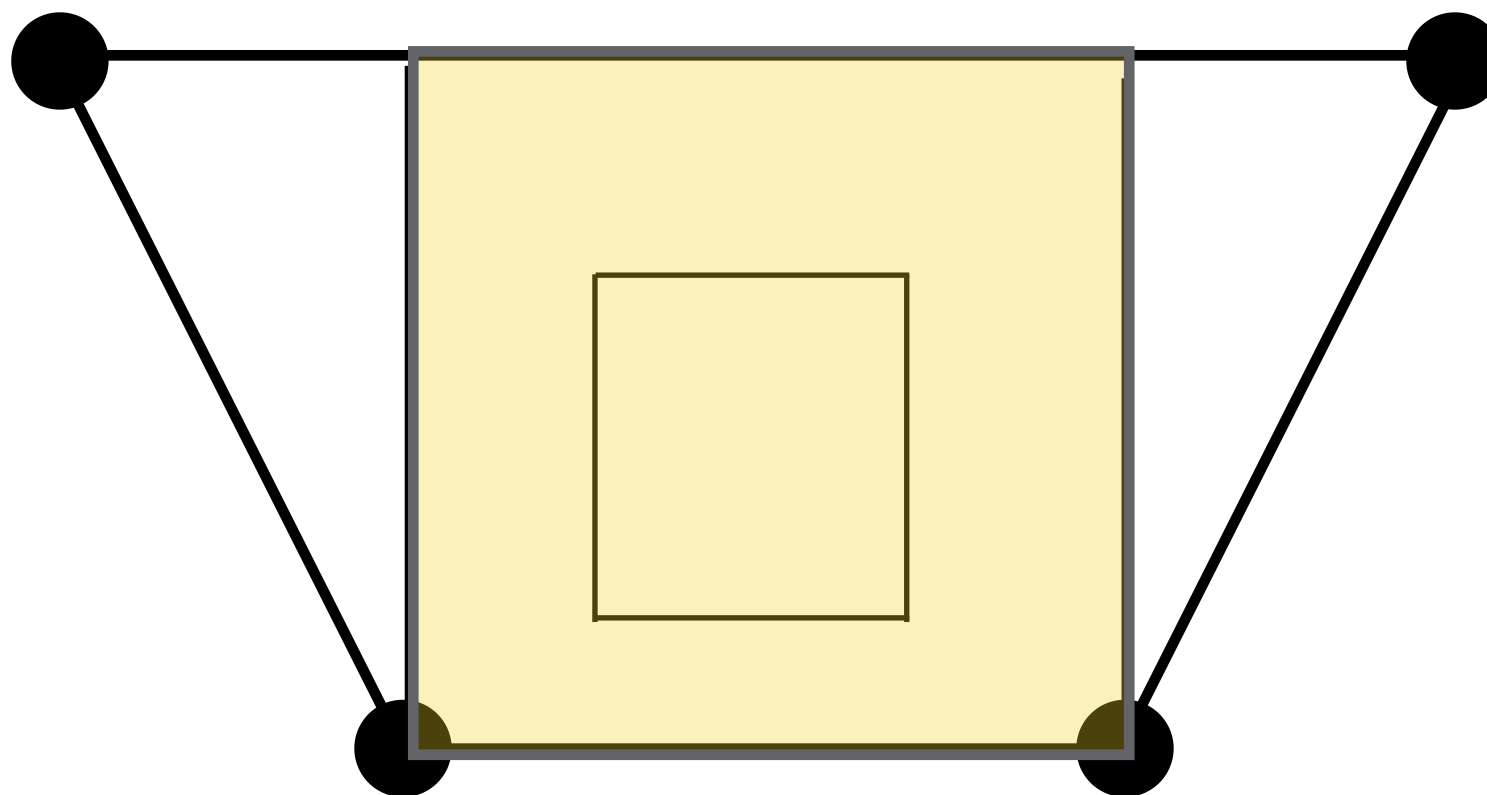




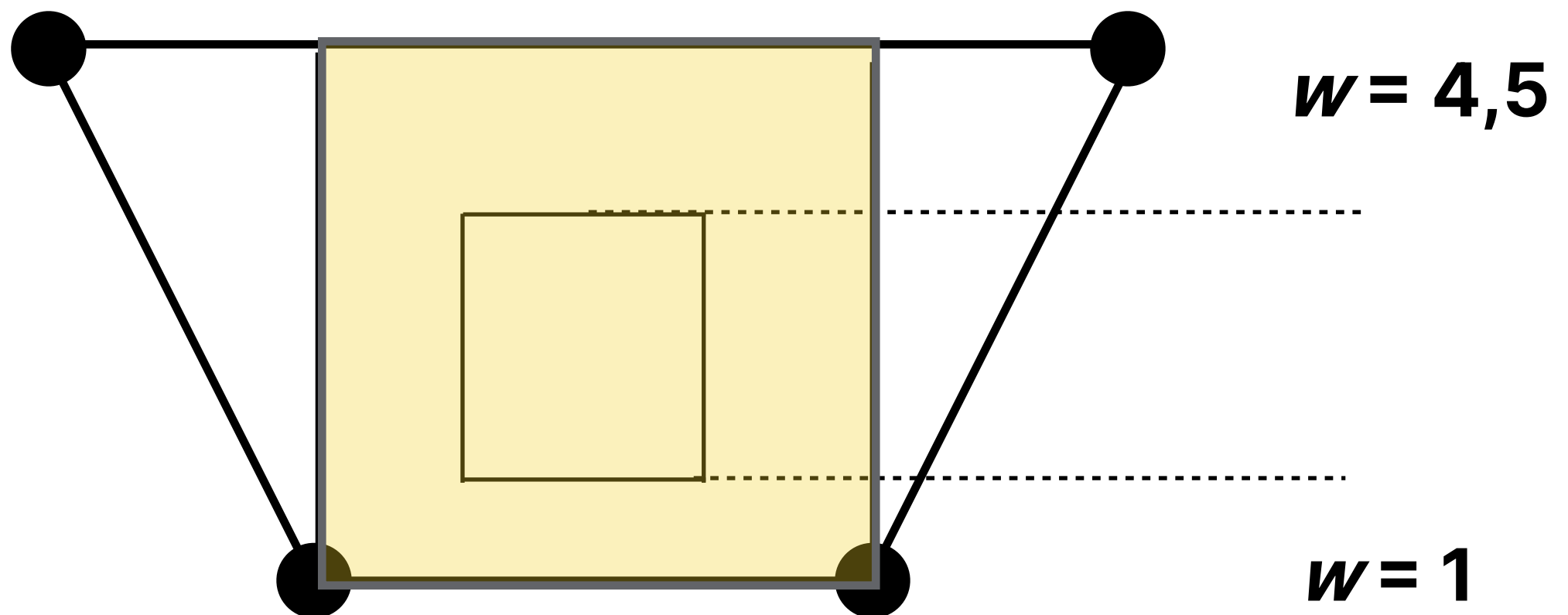
$$\begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



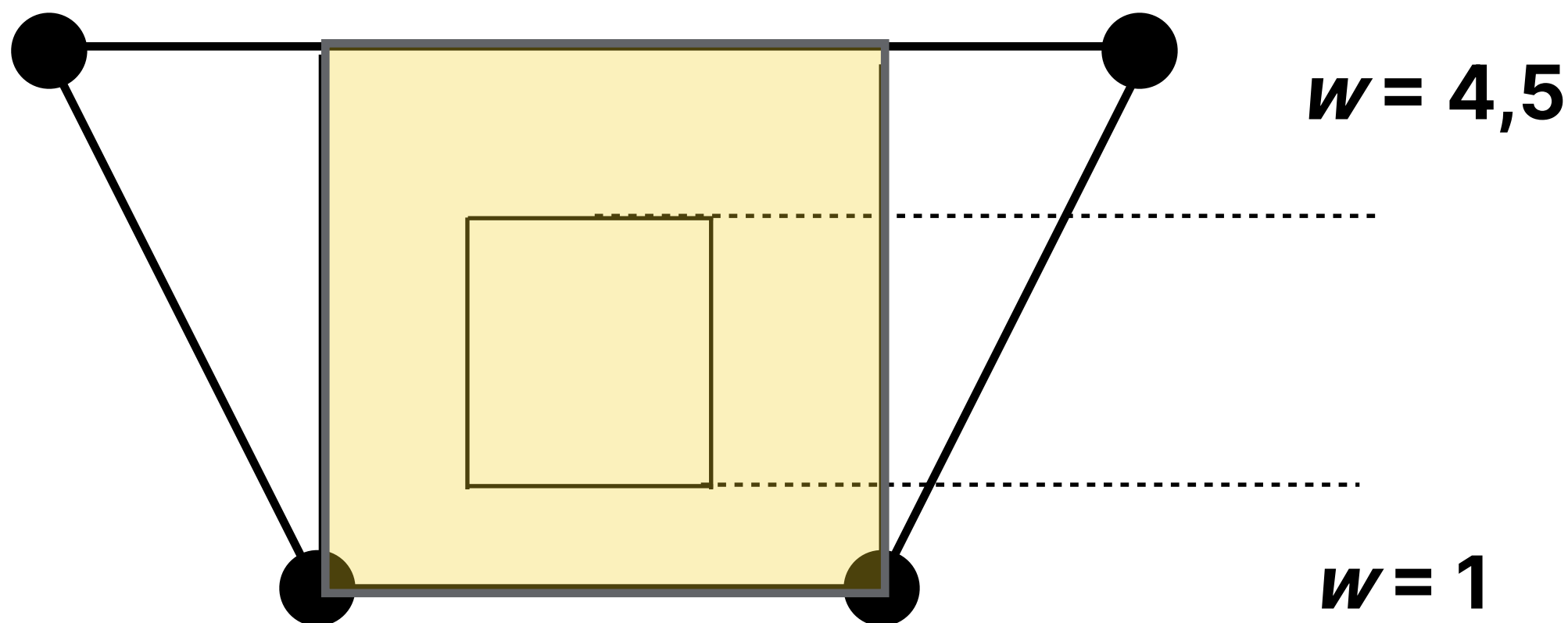
Совмещение усечённой пирамиды с NDC



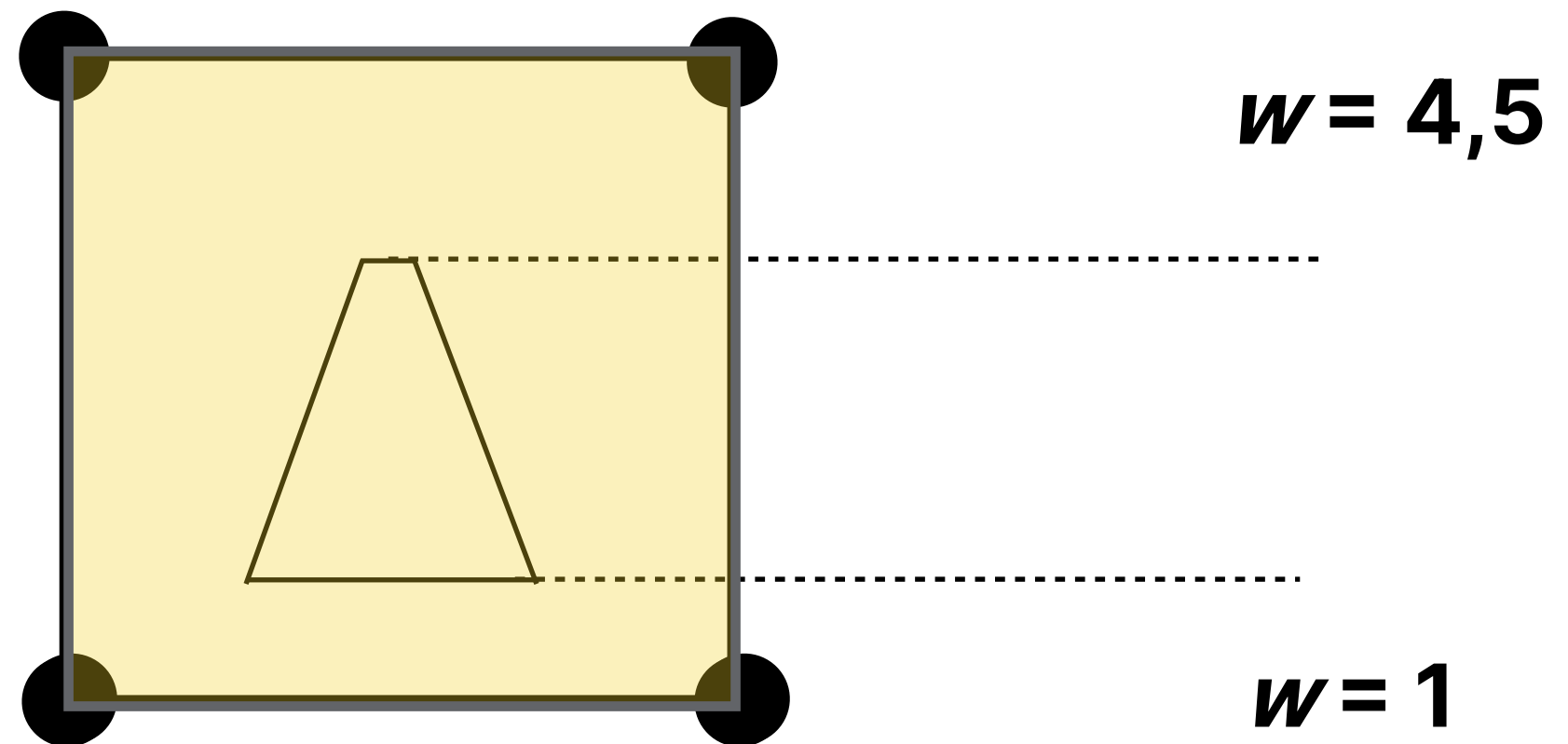
Совмещение усечённой пирамиды с NDC



Совмещение усечённой пирамиды с NDC Деление на w



Совмещение усечённой пирамиды с NDC Деление на w



На самом деле, между пространством наблюдателя
и NDC существует еще одно —
пространство отсечения (clip space)

На самом деле, между пространством наблюдателя и NDC существует еще одно —
пространство отсечения (clip space)

Умножение на матрицу проекции переводит координаты из пространства наблюдателя в пространство отсечения

На самом деле, между пространством наблюдателя и NDC существует еще одно — **пространство отсечения** (clip space)

Умножение на матрицу проекции переводит координаты из пространства наблюдателя в пространство отсечения

Далее в пространстве отсечения выполняется, собственно, отсечение по правилам:

$$-w \leq x \leq w$$

$$-w \leq y \leq w$$

$$-w \leq z \leq w$$

На самом деле, между пространством наблюдателя и NDC существует еще одно — **пространство отсечения** (clip space)

Умножение на матрицу проекции переводит координаты из пространства наблюдателя в пространство отсечения

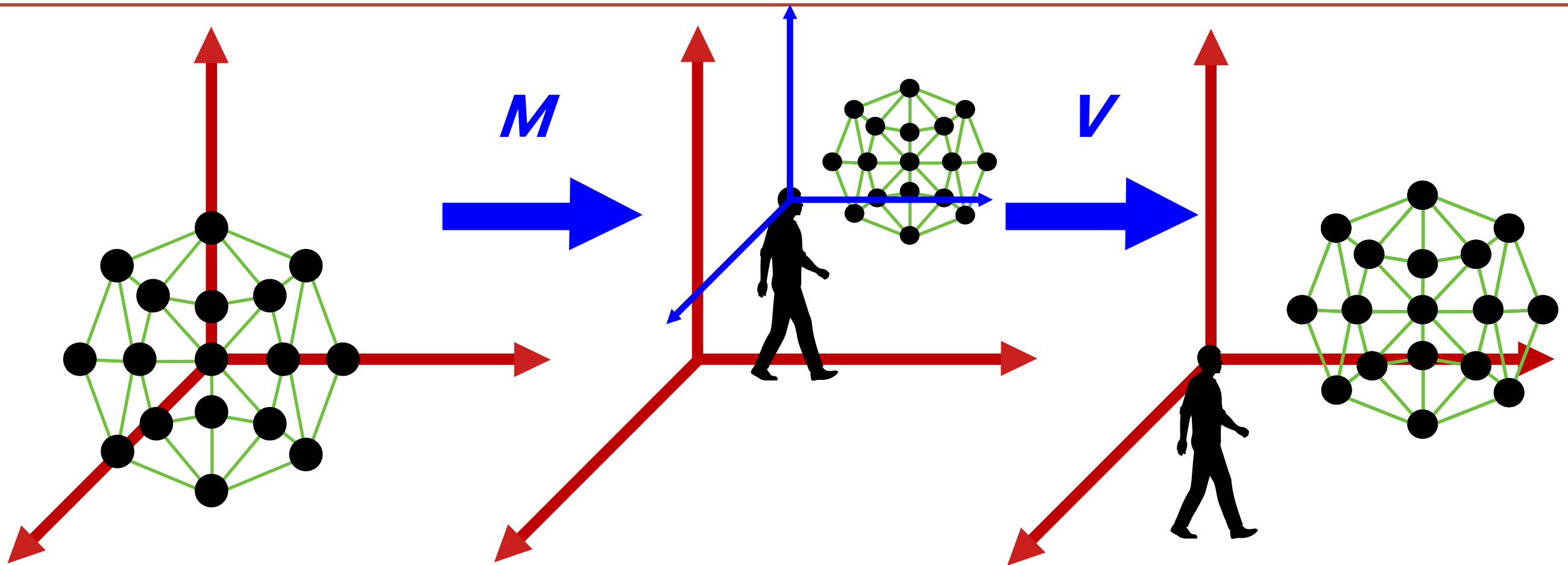
Далее в пространстве отсечения выполняется, собственно, отсечение по правилам:

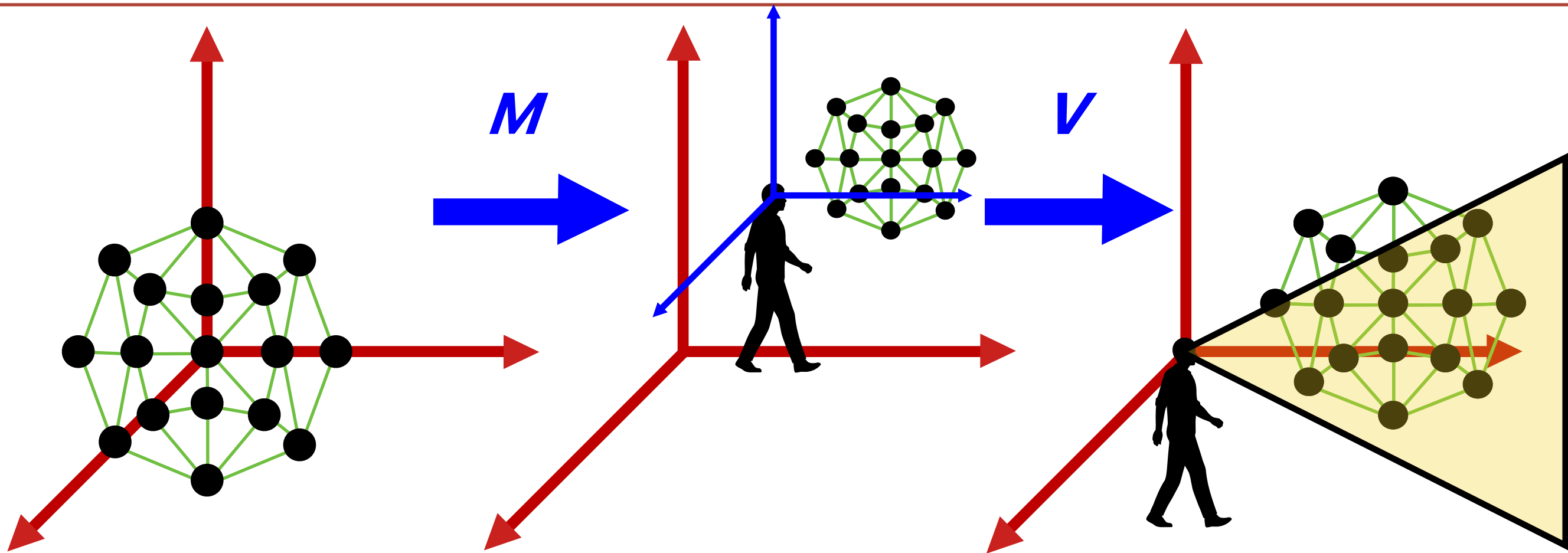
$$-w \leq x \leq w$$

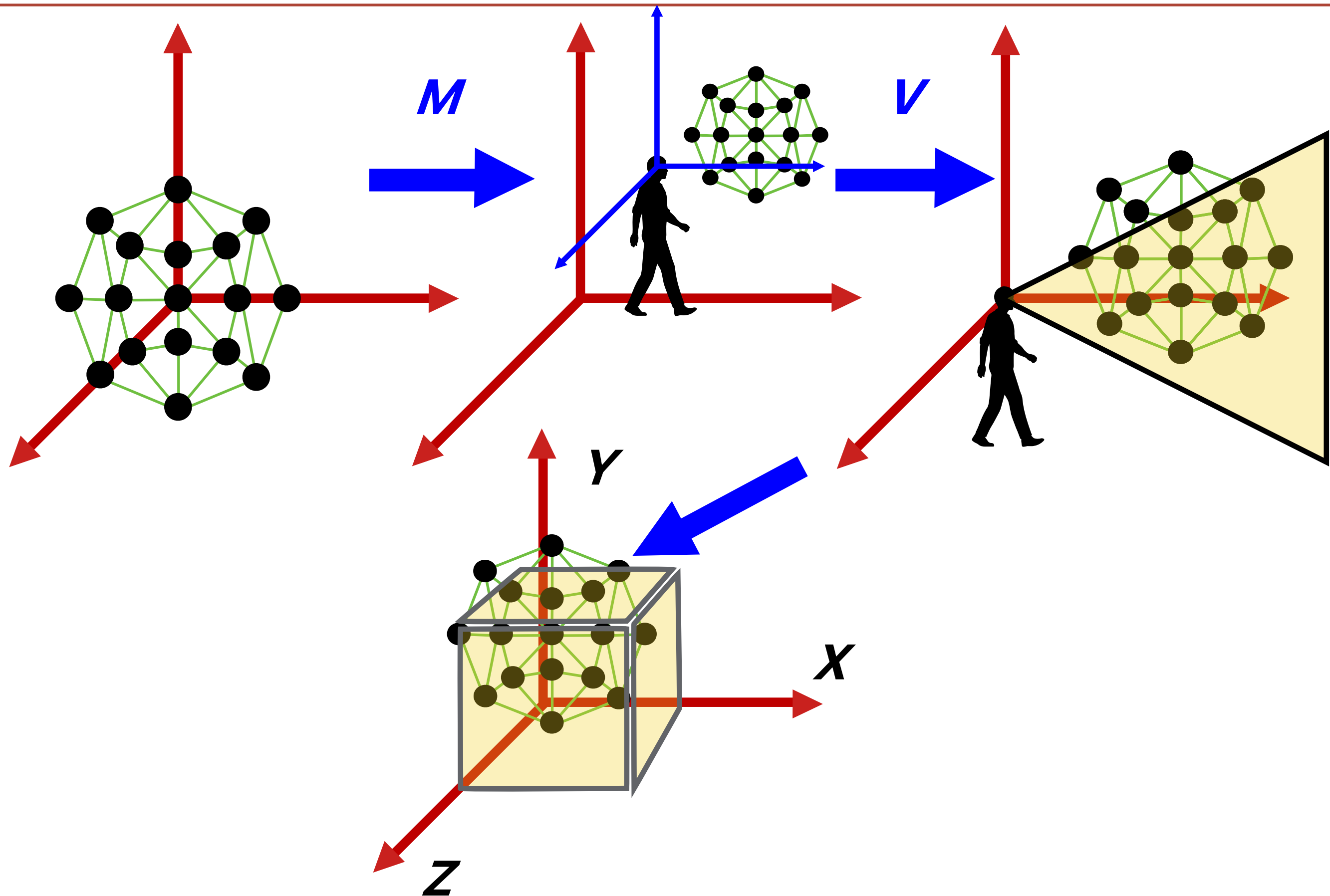
$$-w \leq y \leq w$$

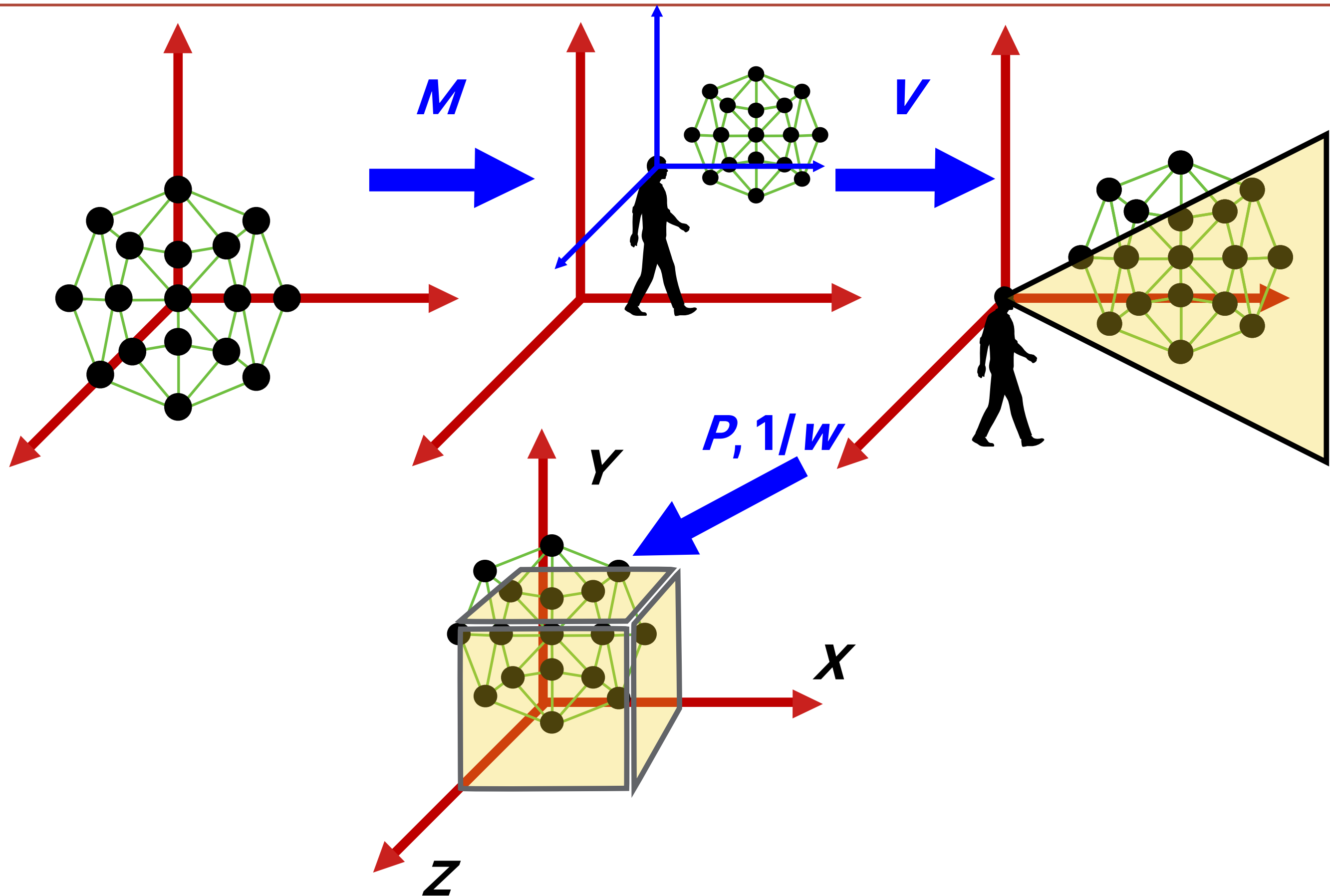
$$-w \leq z \leq w$$

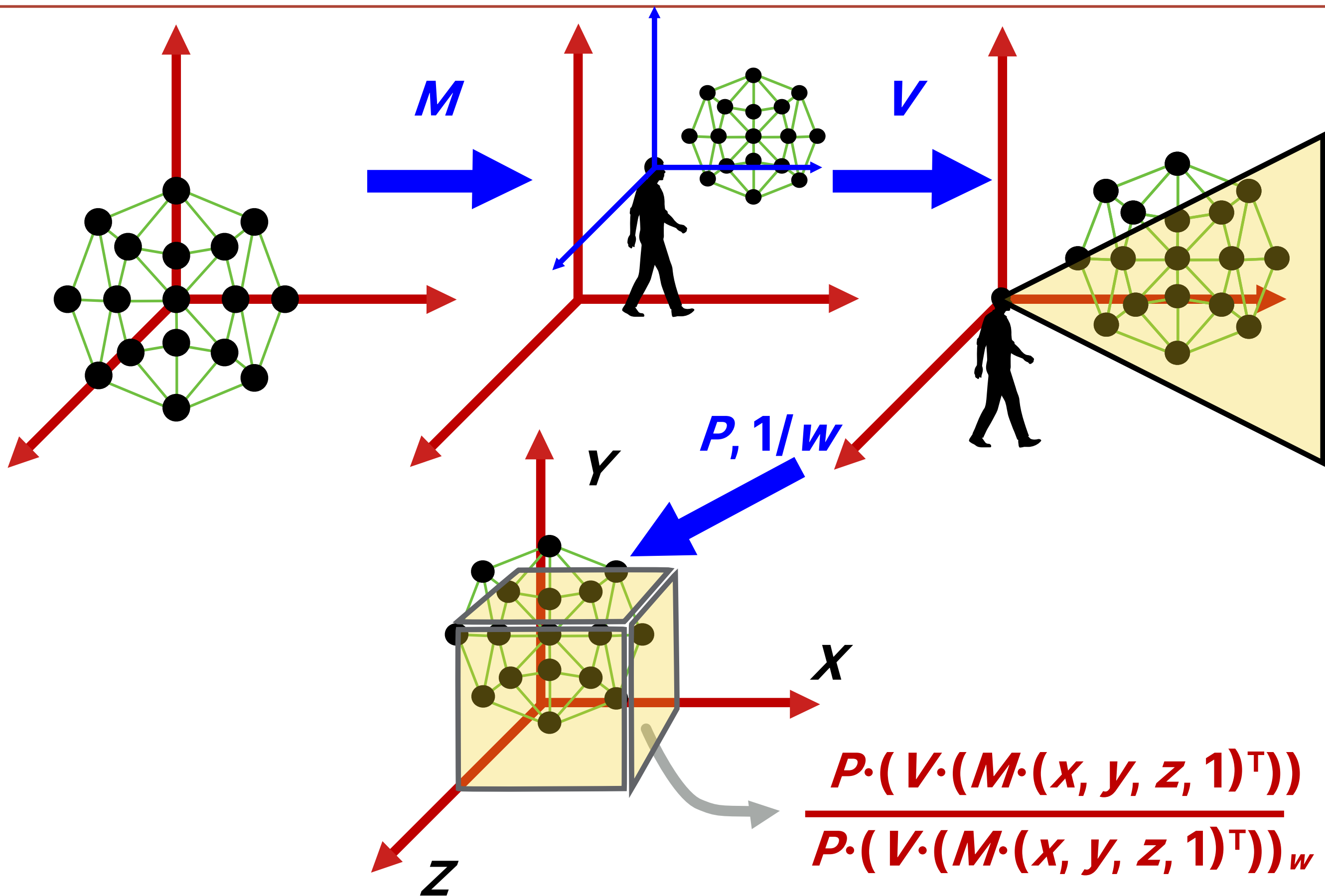
После чего происходит деление на w и переход к NDC











В КГ для хранения преобразований часто используется паттерн Model-View-Projection (MVP):

$$v' = (P \cdot V \cdot M) \cdot v,$$

v' – координаты вершины, передаваемые системе для растеризации

v – исходные координаты вершины

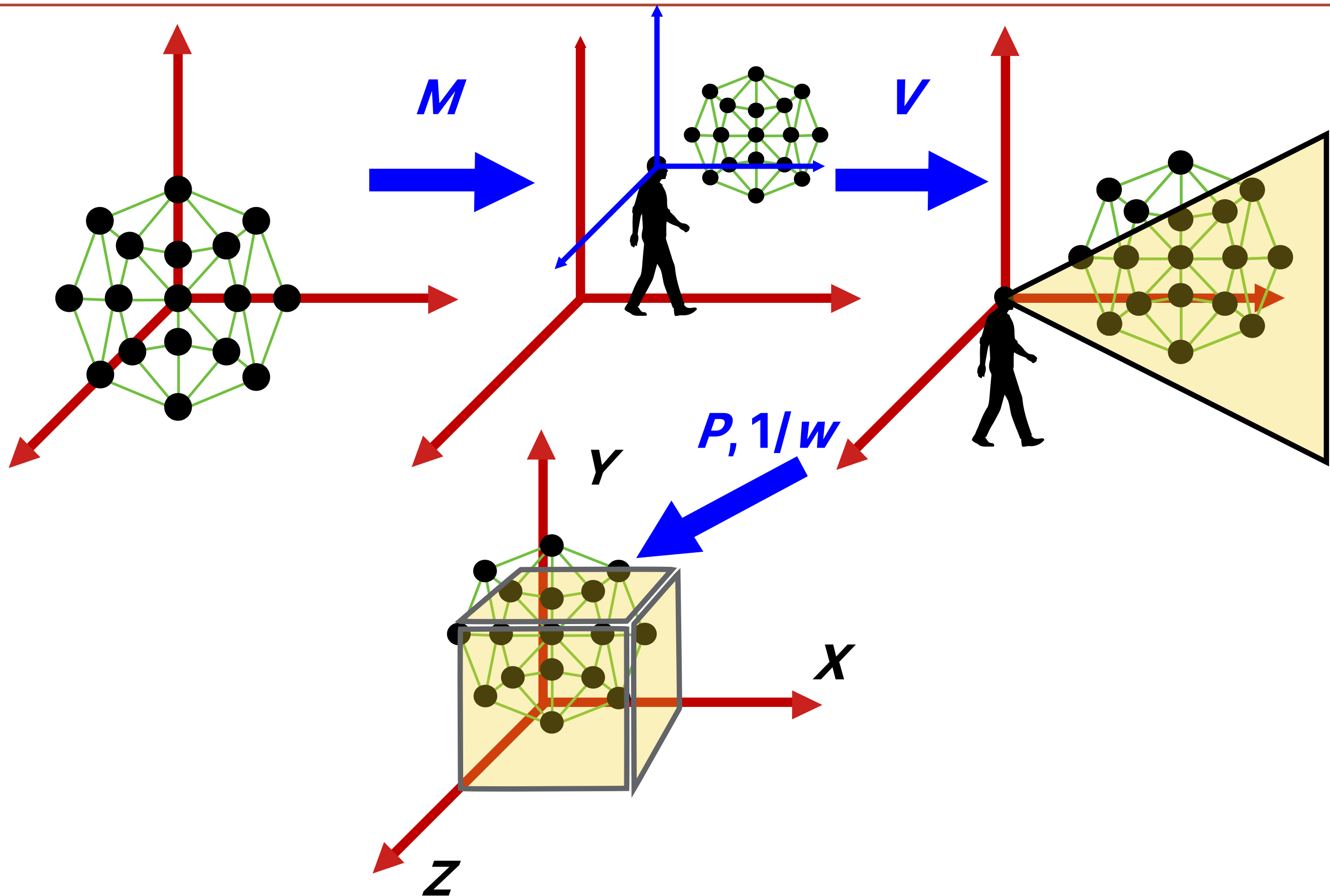
P – матрица проекции

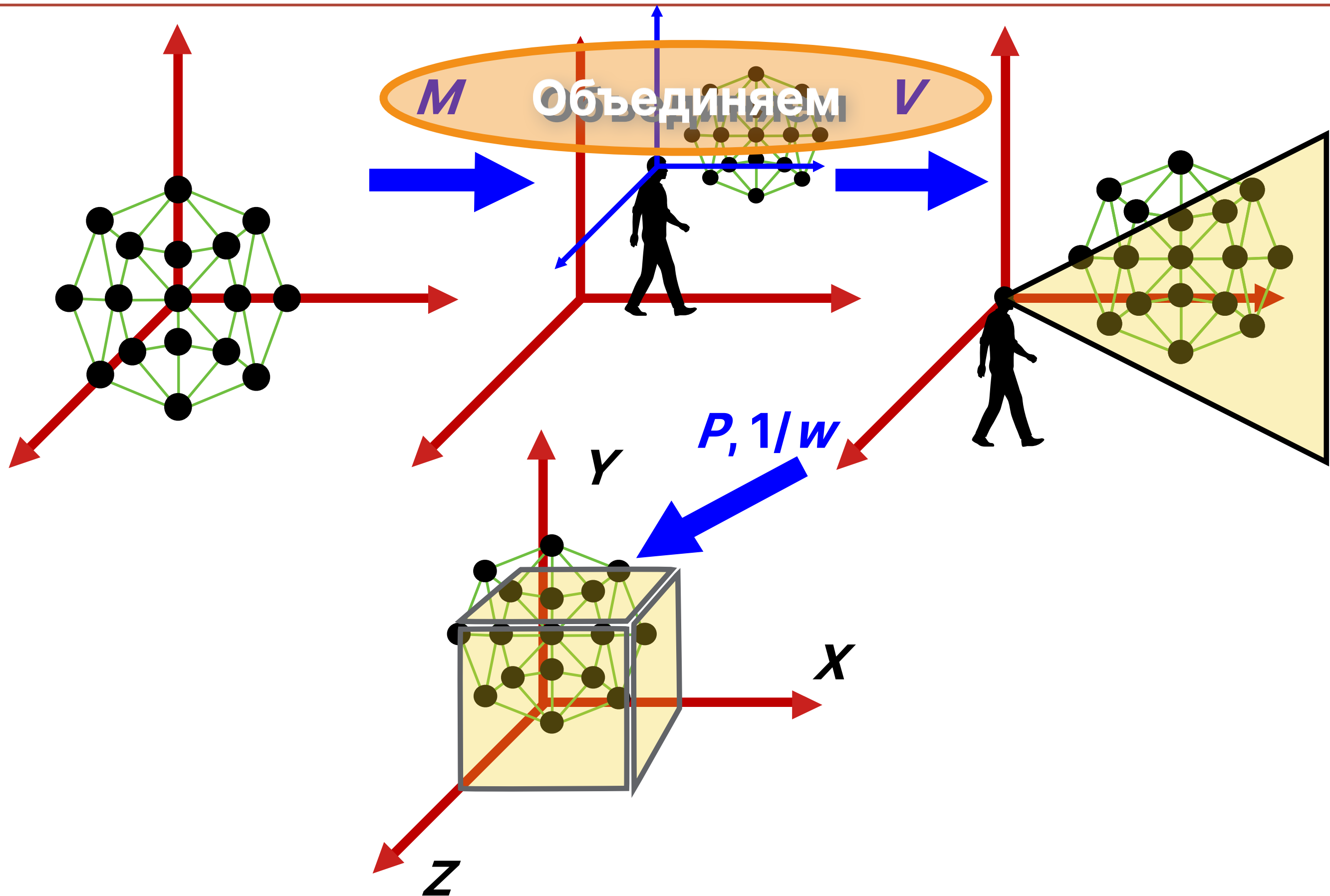
V – матрица вида (преобразование камеры)

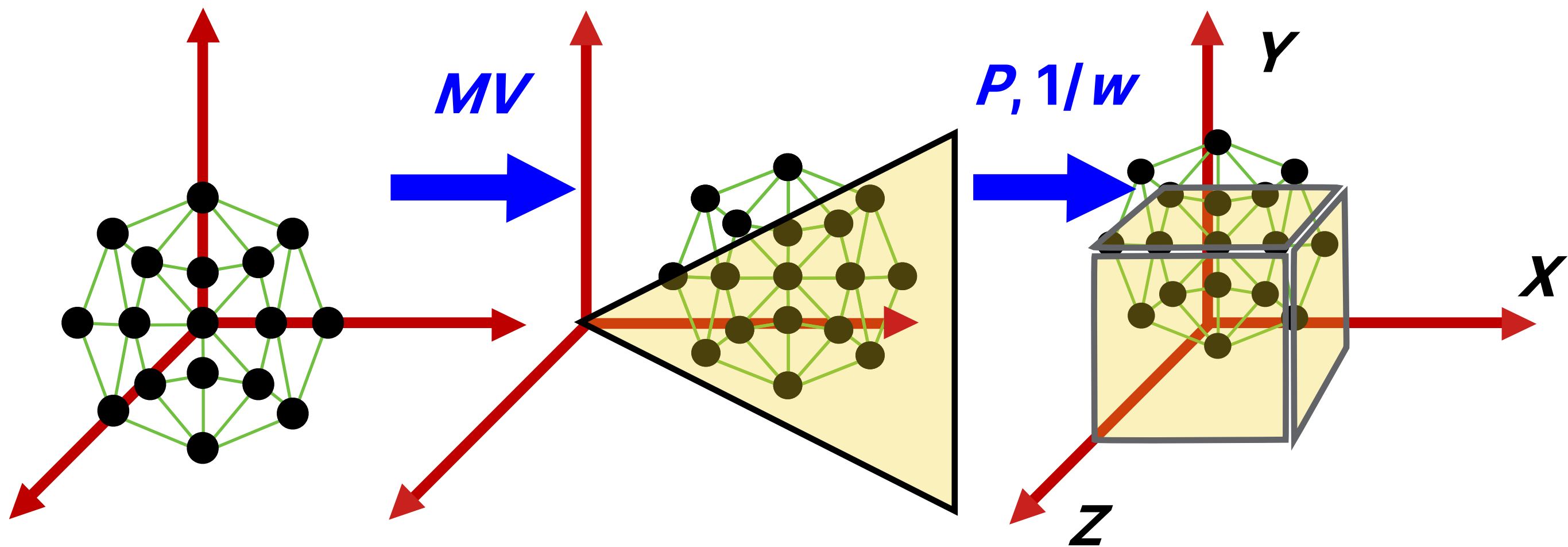
M – матрица модели (преобразование размещения объекта на сцене)

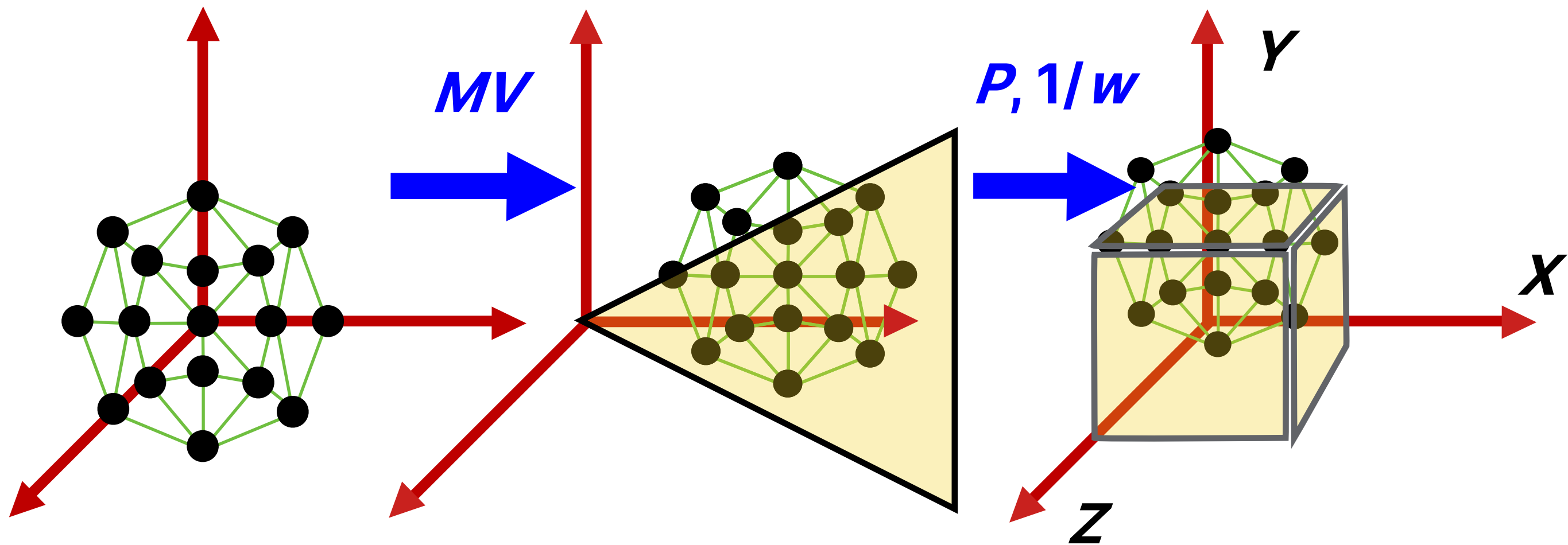
$$M = M_{\text{родителя}} \cdot M_{\text{объекта}}$$

- **Камера** – это псевдообъект в трёхмерном пространстве, характеризующий положение наблюдателя
- **Камера** – лишь полезная метафора, на низком уровне она выражена матричным преобразованием, математически ничем не отличающимся от всех остальных
- Часто преобразование камеры является лишь аффинным
- В связи с этим, иногда преобразование камеры не хранят отдельно, а «смешивают» его с преобразованием размещения, получая матрицу, которую принято называть ModelView (в «олдскульном» OpenGL было именно так)

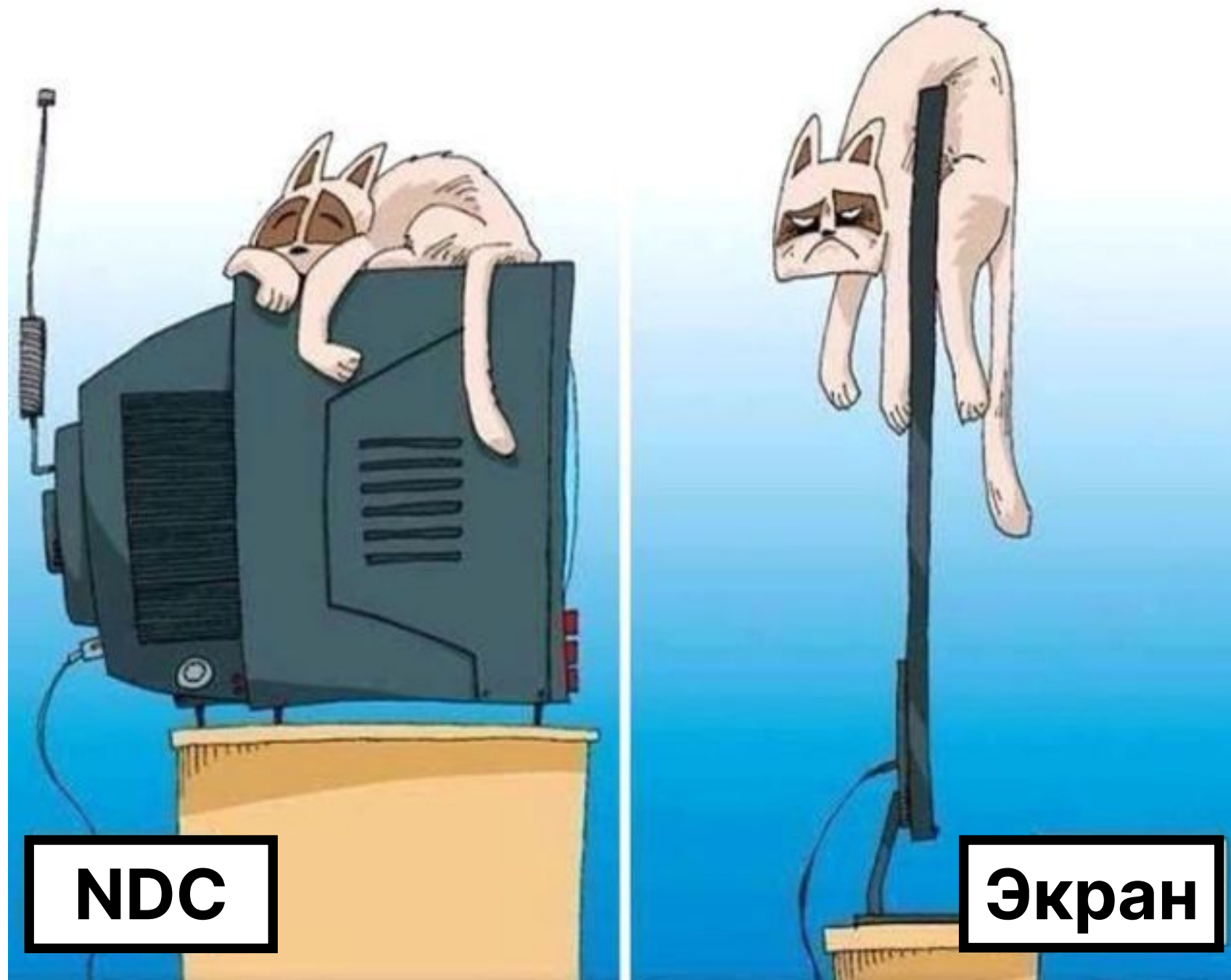






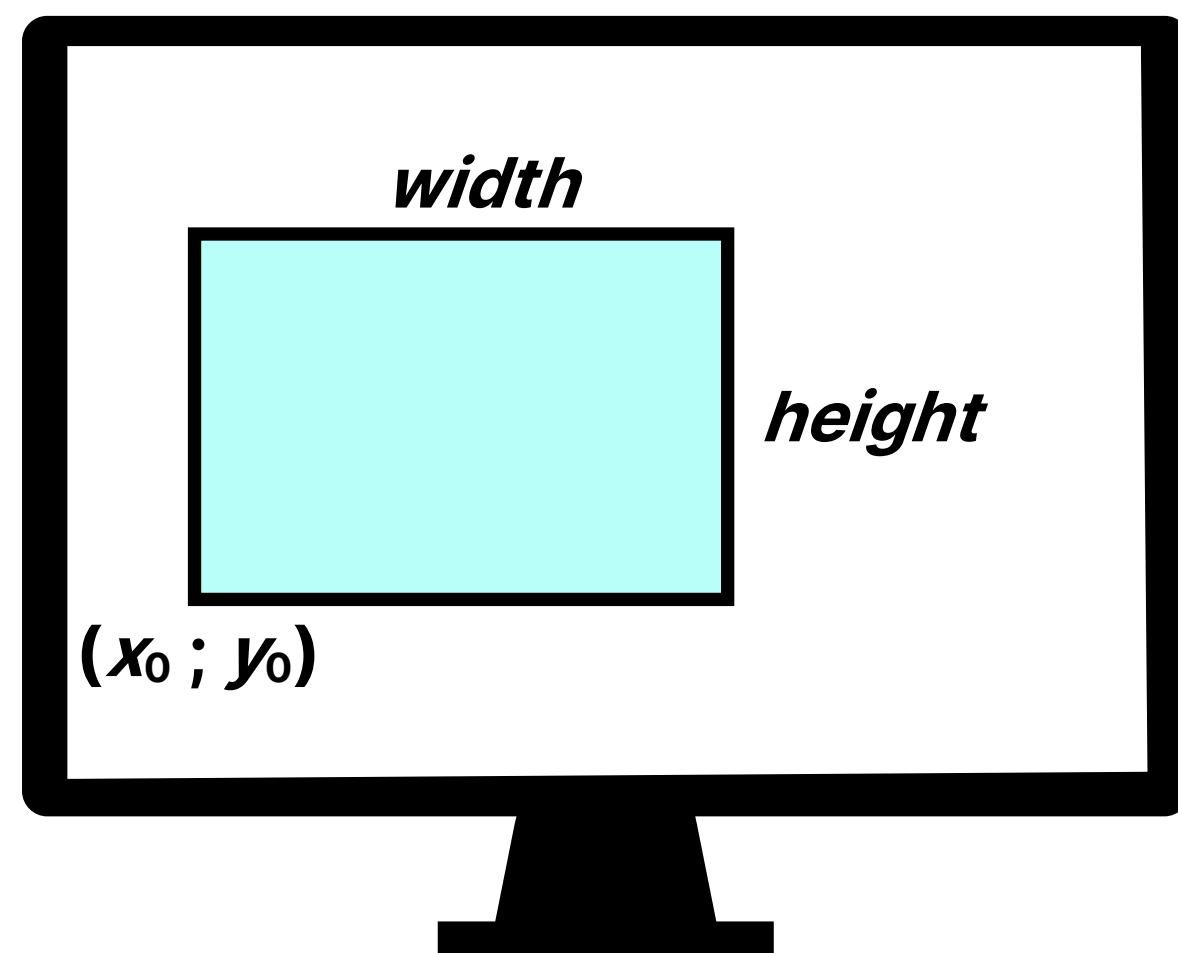
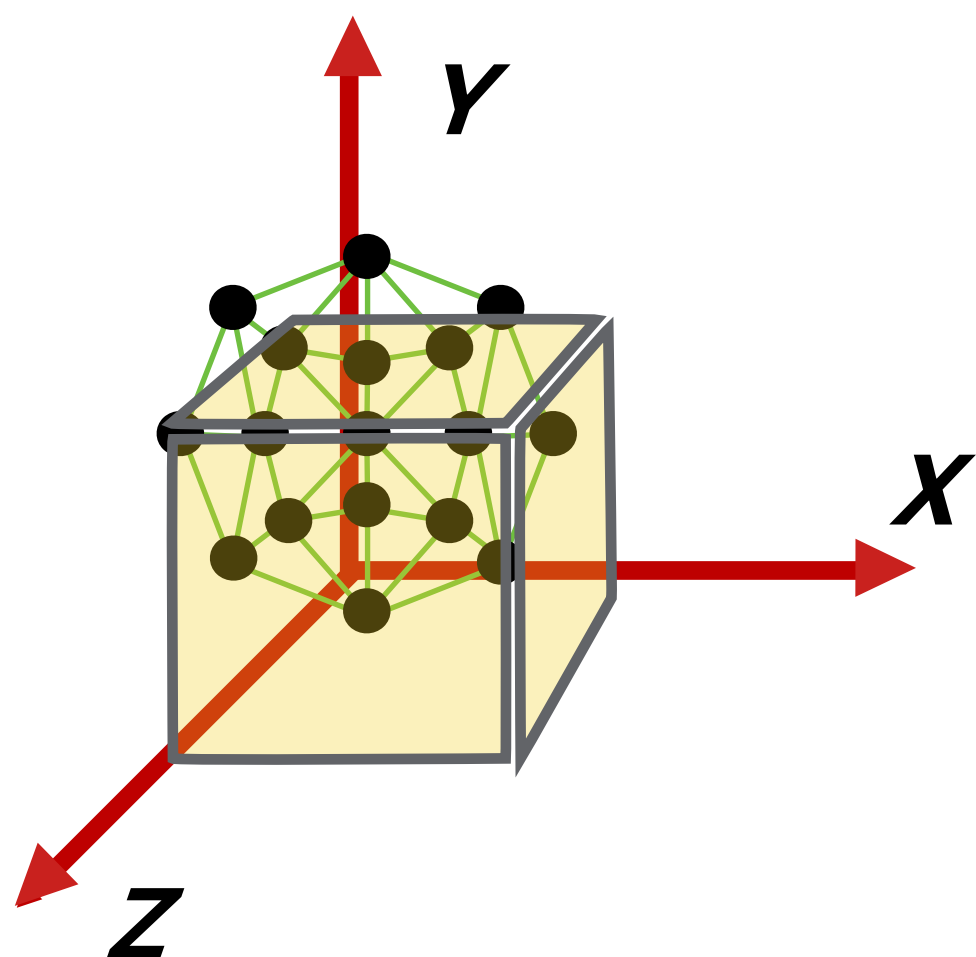


Но и это ещё не всё!

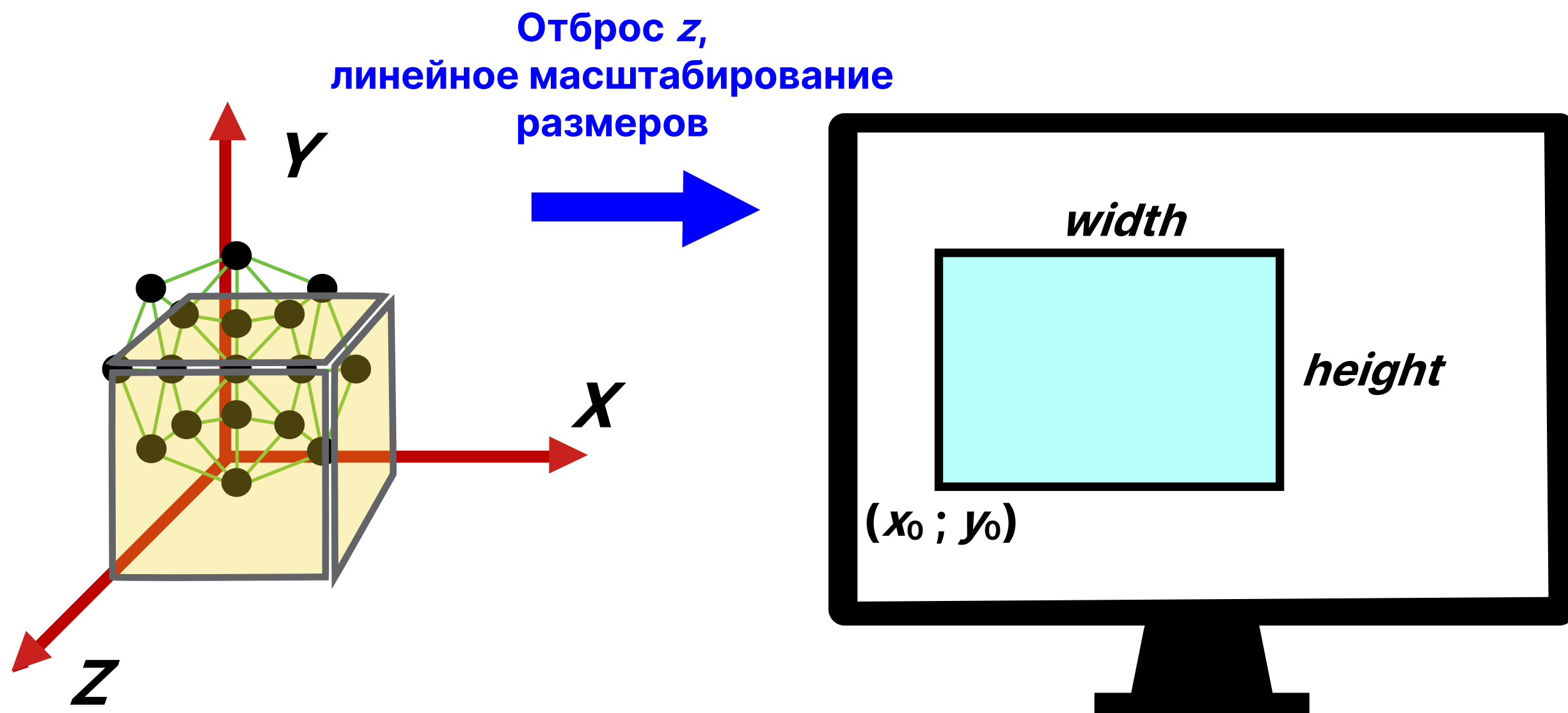


NDC – объёмная конструкция, а экран – плоский

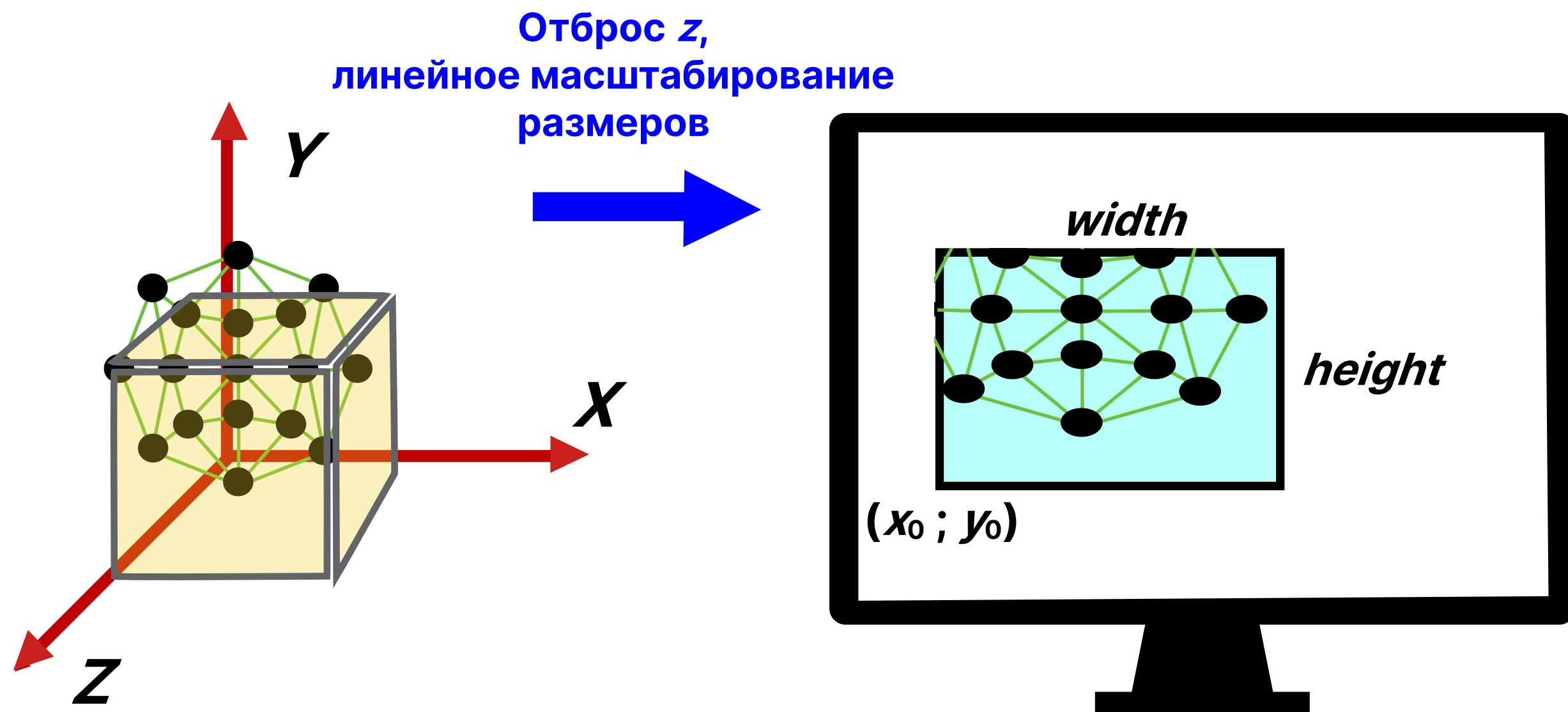
Переход от NDC к экрану осуществляется
преобразованием порта просмотра
(View Port Transform)



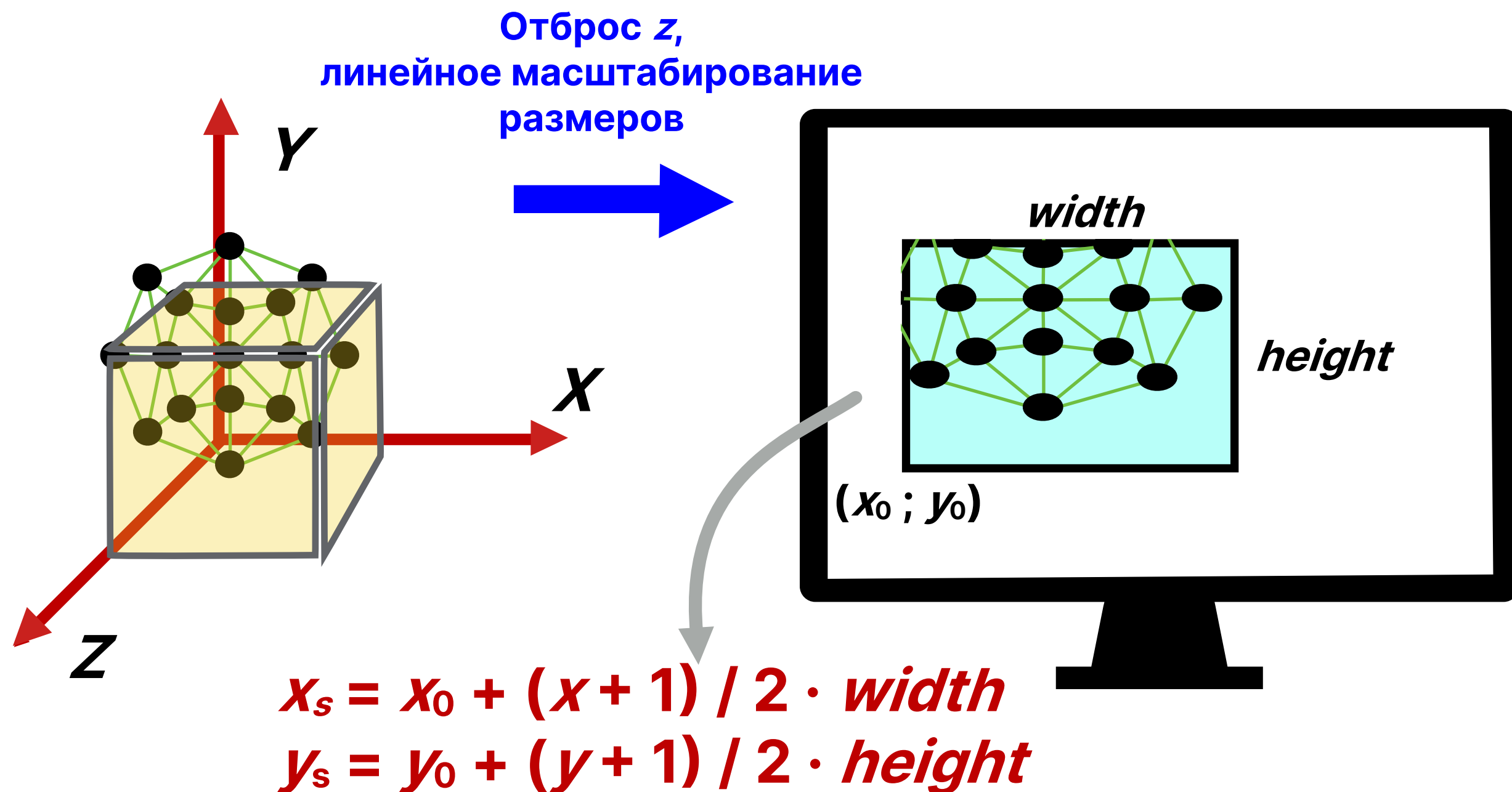
Переход от NDC к экрану осуществляется
преобразованием порта просмотра
(View Port Transform)



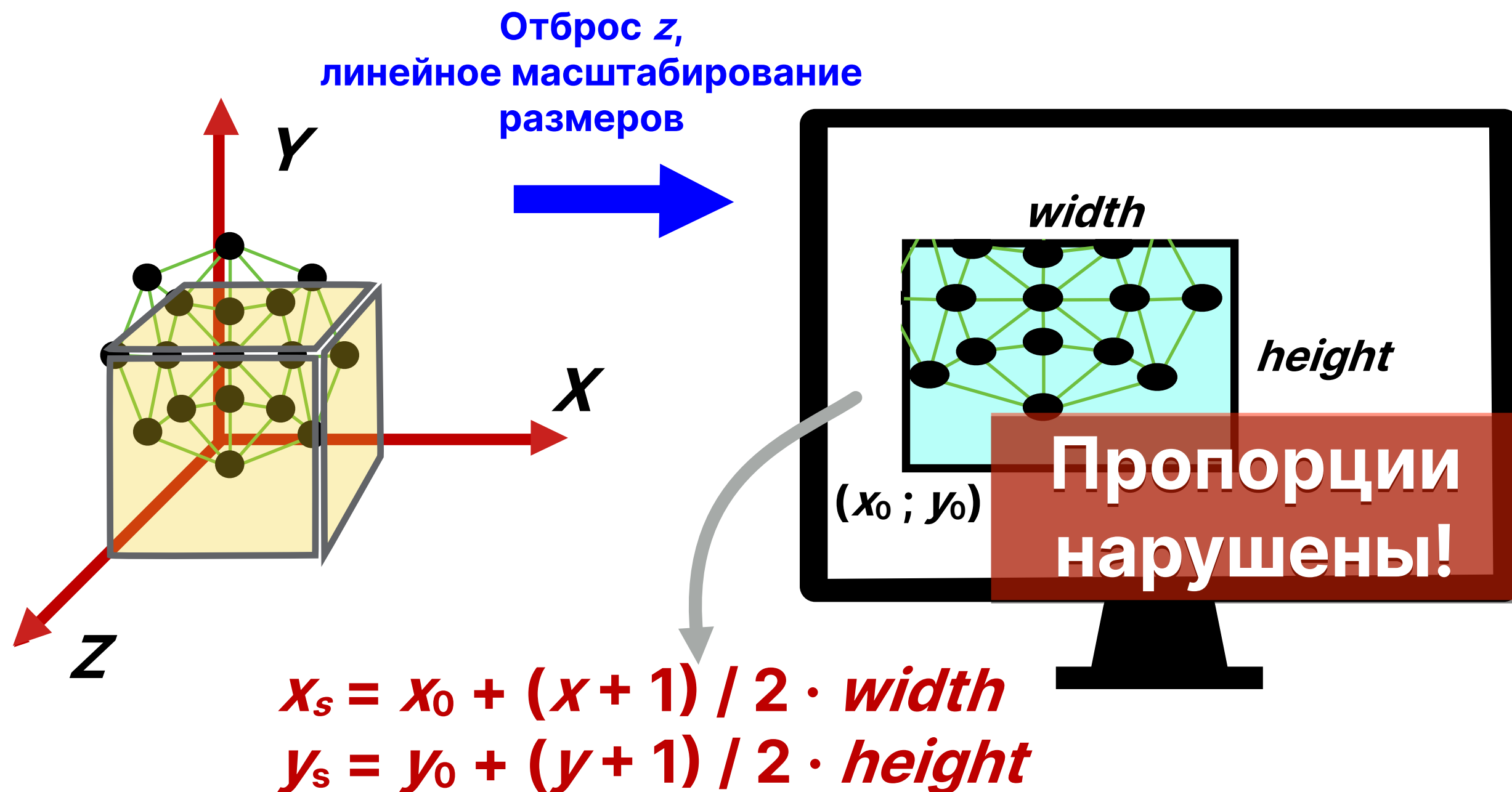
Переход от NDC к экрану осуществляется
преобразованием порта просмотра
(View Port Transform)



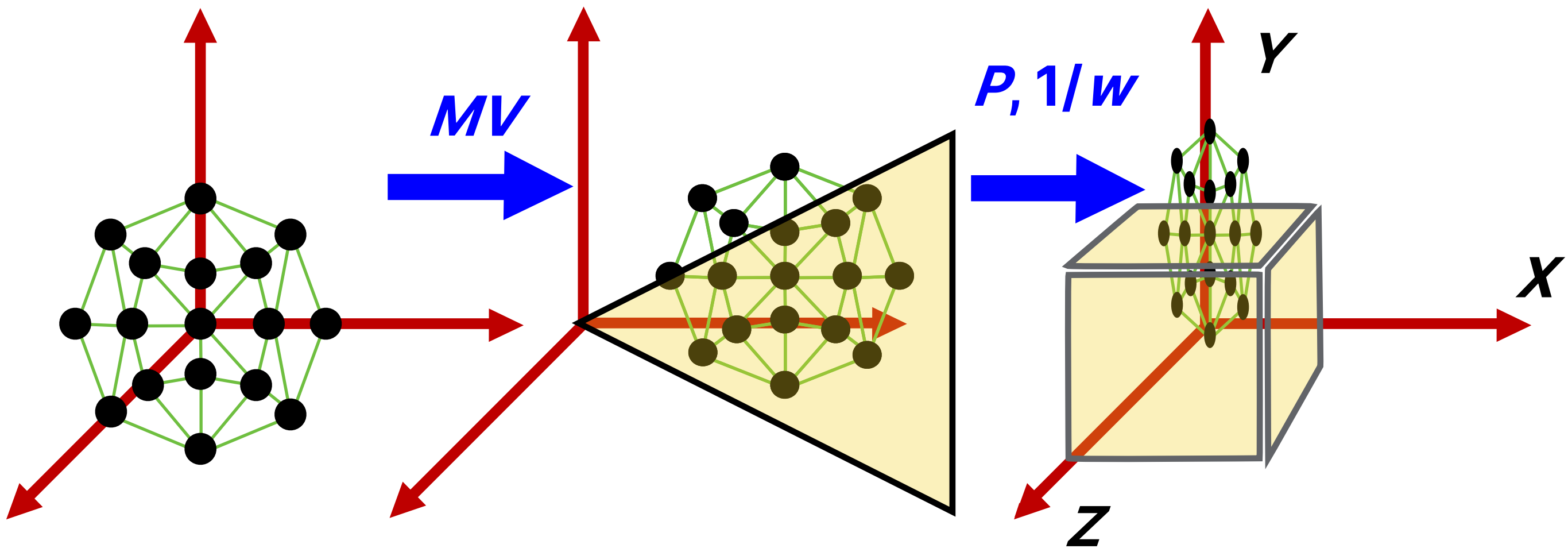
Переход от NDC к экрану осуществляется
преобразованием порта просмотра
(View Port Transform)



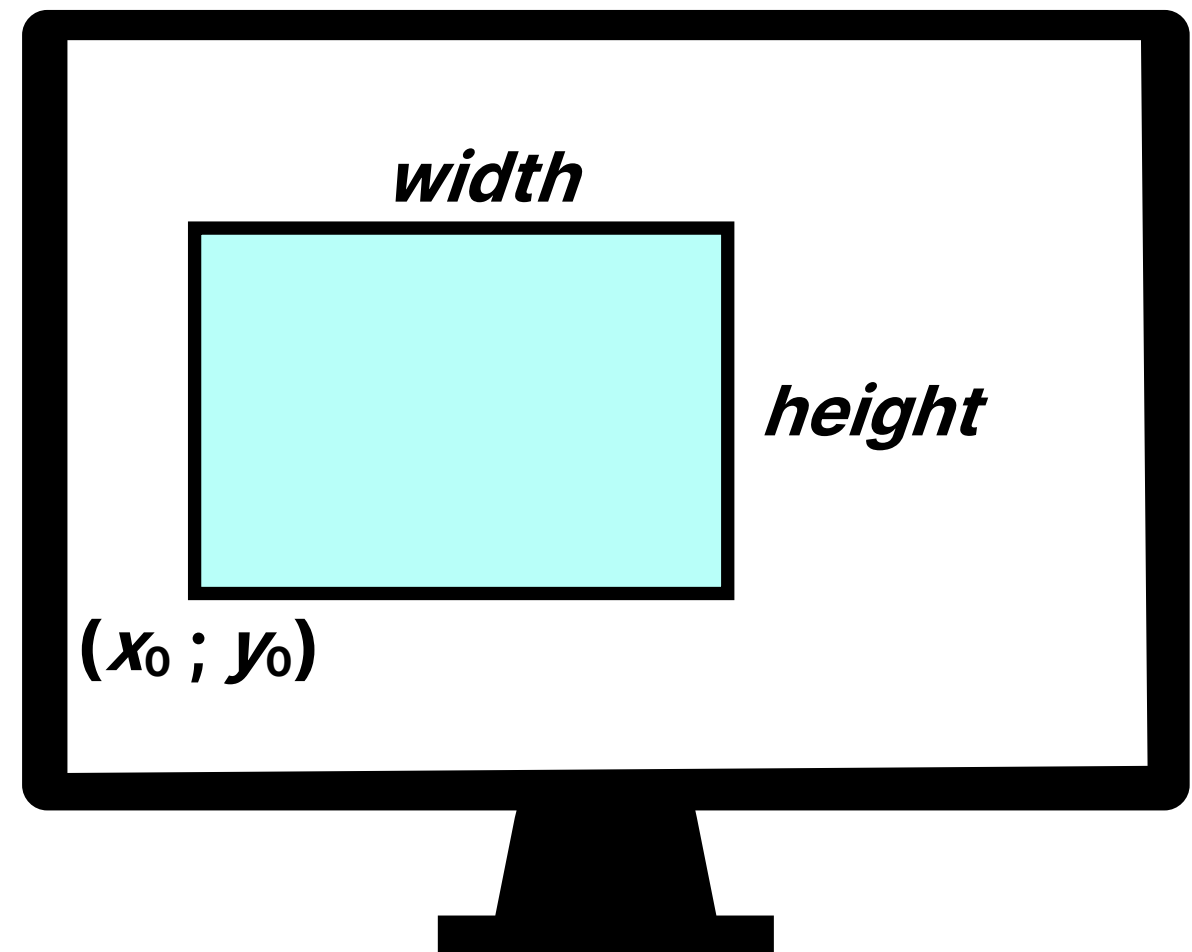
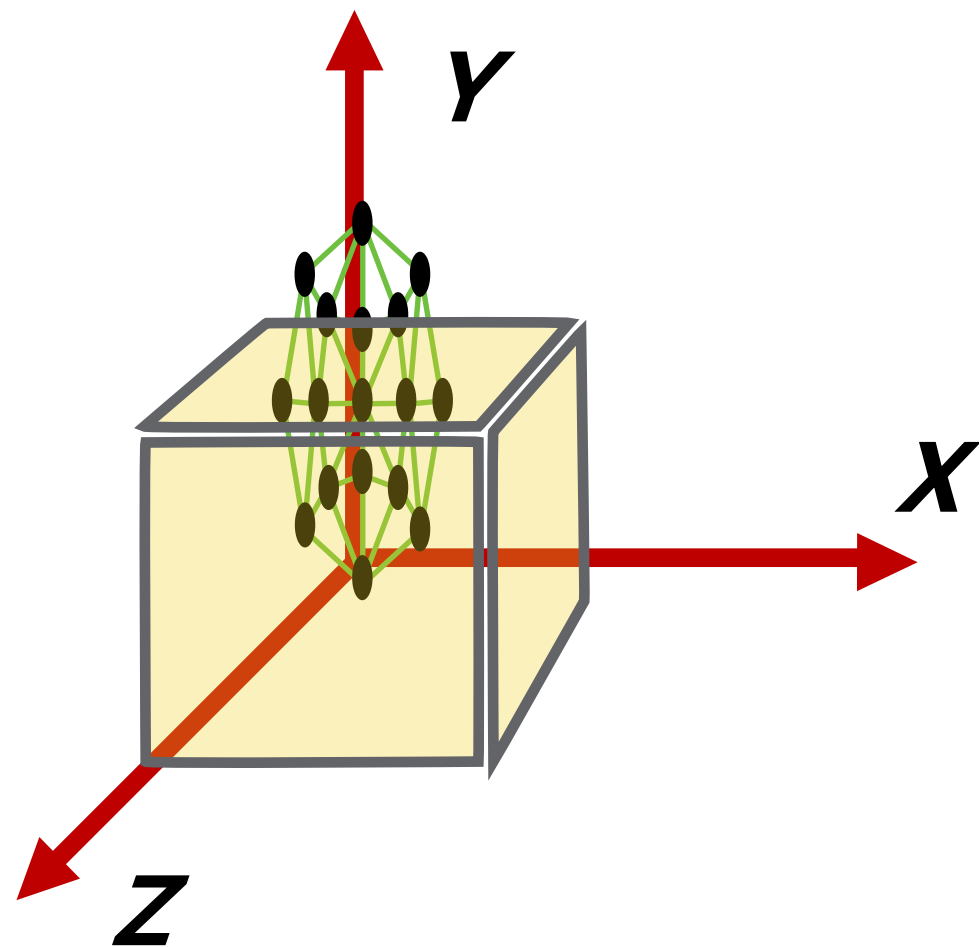
Переход от NDC к экрану осуществляется
преобразованием порта просмотра
(View Port Transform)



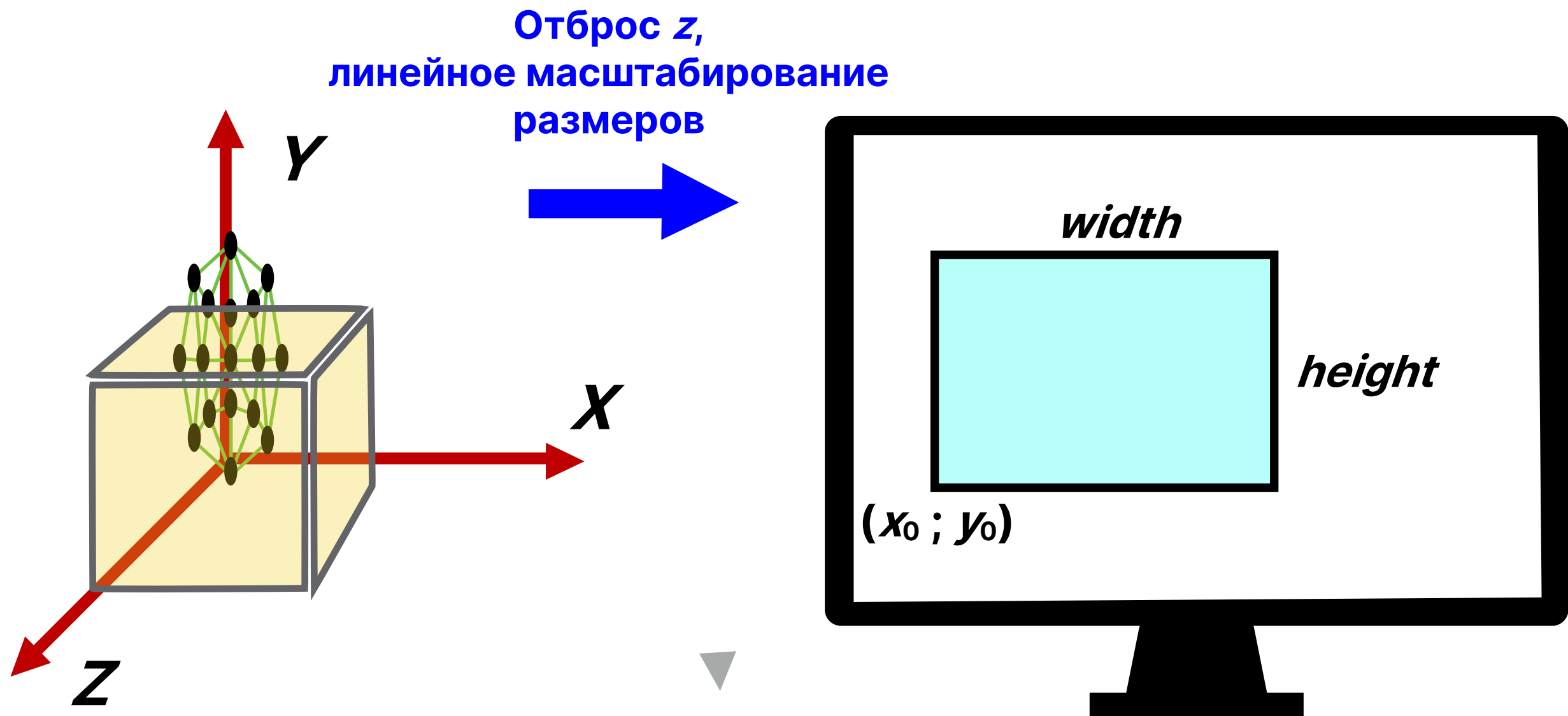
Идея: добавить в матрицу P
дополнительное сжатие в (*width / height*) раз



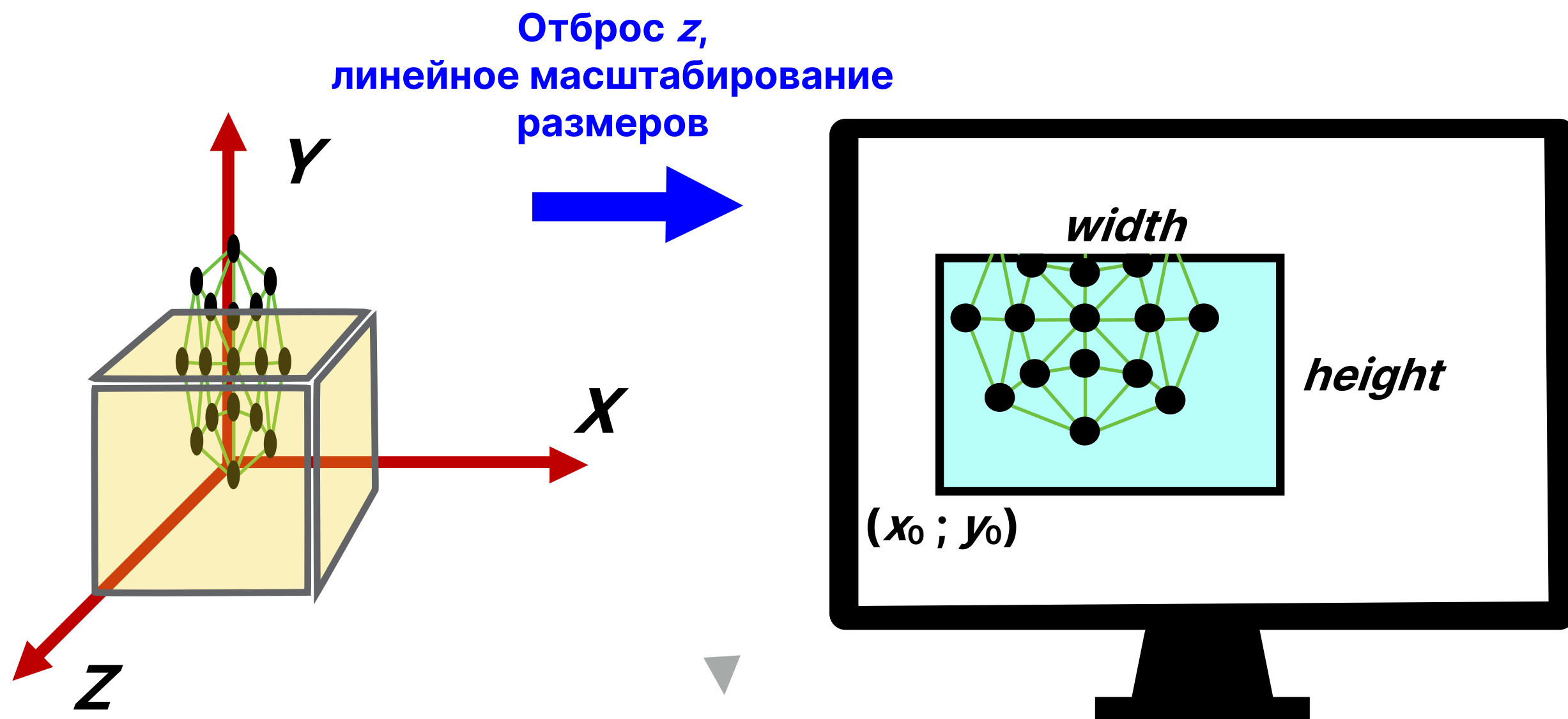
**Растяжение при преобразовании порта просмотра
восстановит пропорции**

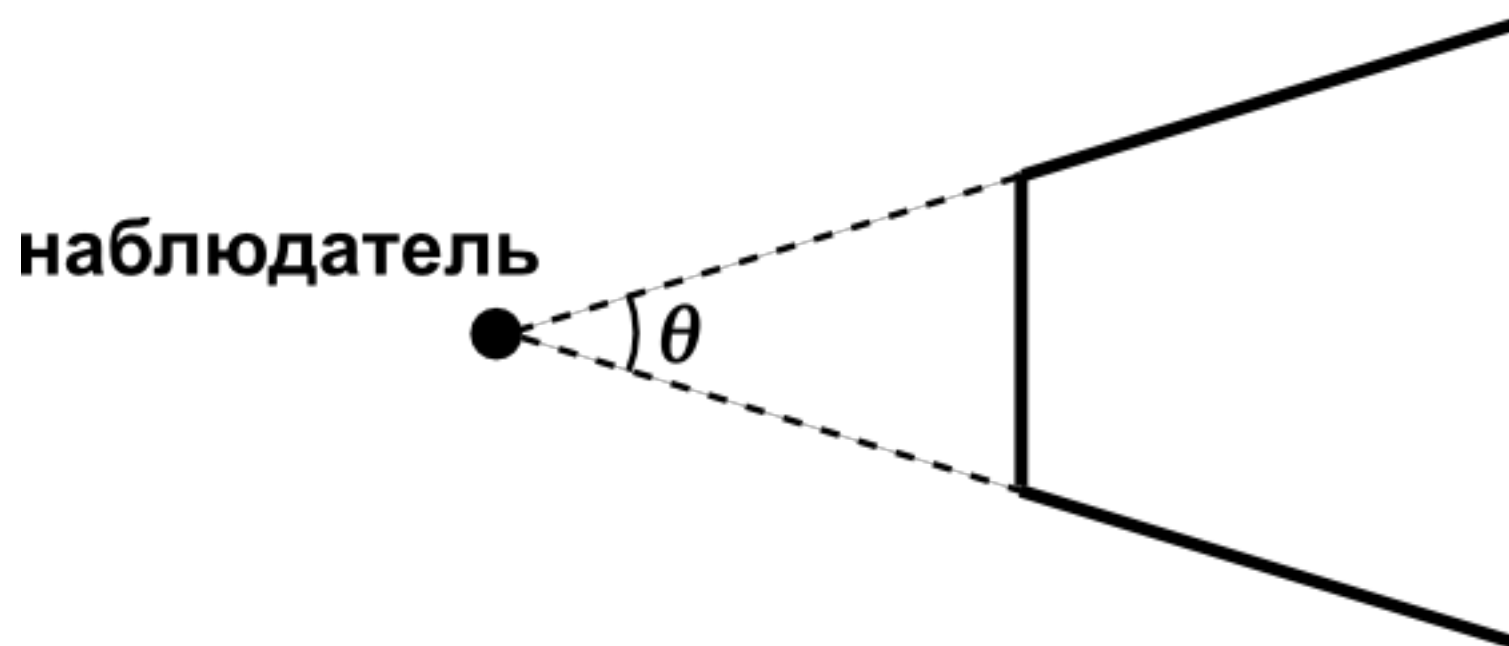


Растяжение при преобразовании порта просмотра восстановит пропорции



Растяжение при преобразовании порта просмотра восстановит пропорции

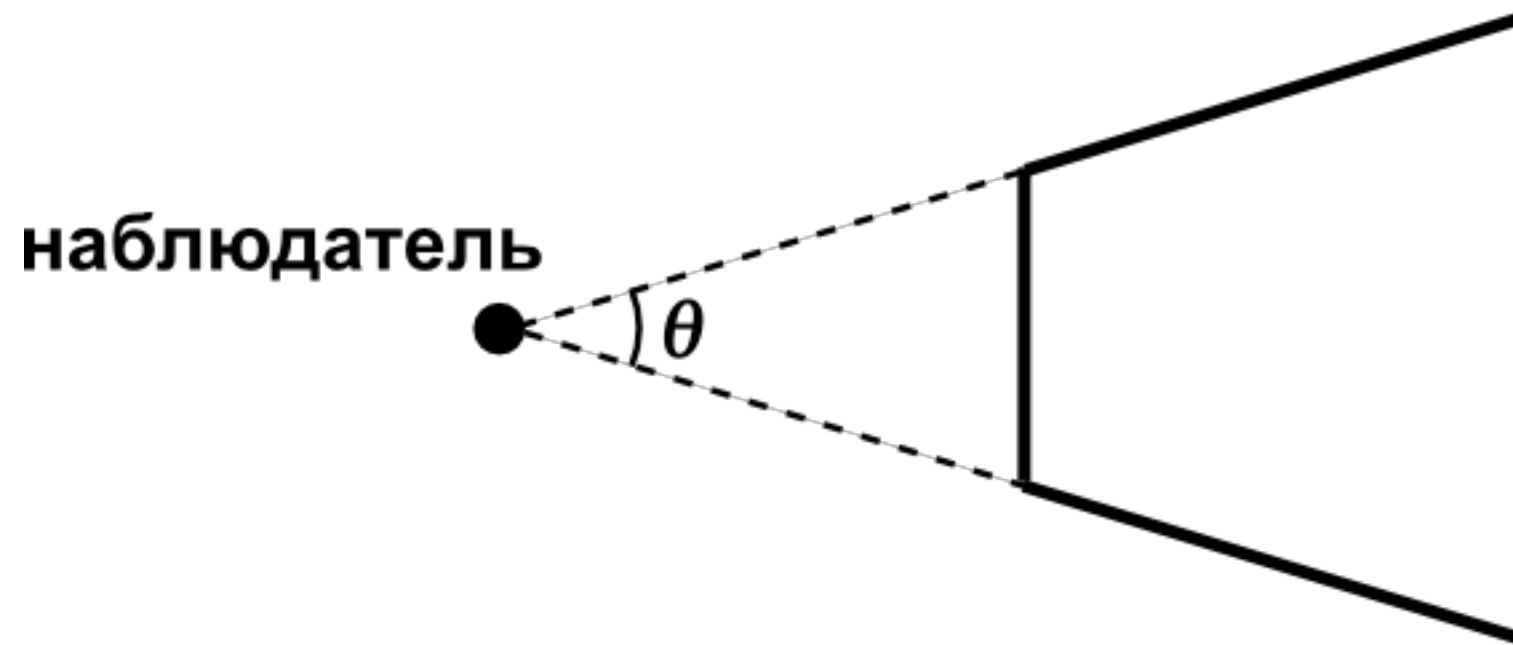




$$-l = r = n \cdot \operatorname{tg} \frac{\theta}{2}$$

$$-b = t = n \cdot \frac{w}{h} \cdot \operatorname{tg} \frac{\theta}{2}$$

$$\begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

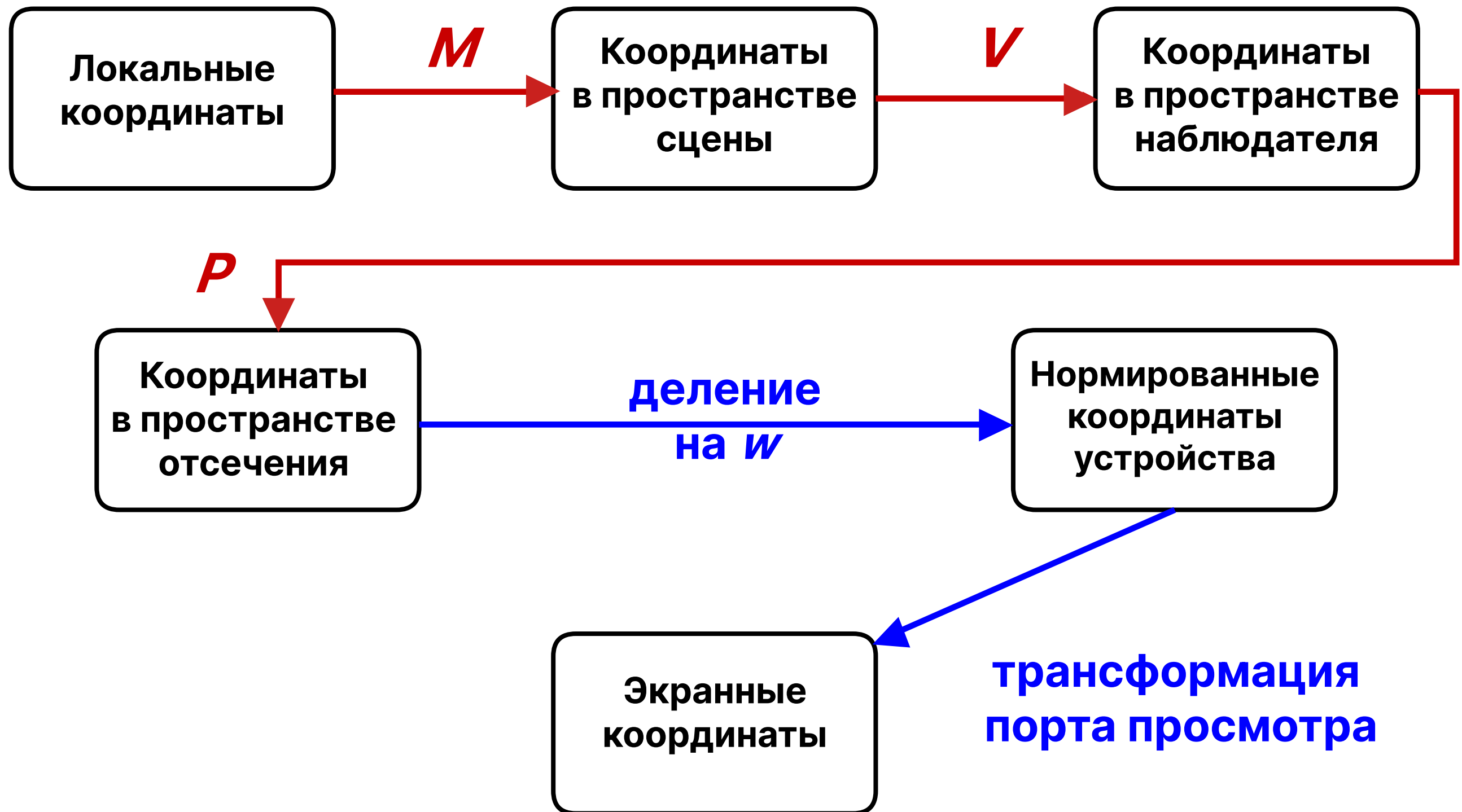


$$-l = r = n \cdot \operatorname{tg} \frac{\theta}{2}$$

$$-b = t = n \cdot \frac{w}{h} \operatorname{tg} \frac{\theta}{2}$$

$$\begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

для
сохранения
пропорций



Выполняется программистом

Выполняется системой